WaferCalibSDK 视觉算法库接口与开发指南

本文档面向自动化机台控制软件、上层 C# (.NET) 视觉检测软件及 C++ 开发者，系统阐述 `WaferCalibSDK.dll` 各核心视觉功能模块的技术原理、调用流程、接口参数及示例代码。

目录

- 一、 通用约定与基础规范
- 1.1 图像格式与内存对齐
- 1.2 SDK 状态返回码
- 1.3 C# P/Invoke 原生互操作基类定义
- 1.4 工业相机采集内存传递规范
- 二、 暗场背景降噪功能 (NoiseReduction)
- 2.1 业务场景与技术原理
- 2.2 核心接口说明 (Wafer_CreateDarkFrameTemplate)
- 2.3 C# 调用代码示例
- 2.4 实测效果展示
- 三、 四十字 Mark 对角线交点定位功能 (MarkCenter)
- 3.1 业务场景与技术原理
- 3.2 核心接口说明 (Wafer_FindFourCrossMarkCenter)
- 3.3 C# 调用代码示例
- 3.4 实测效果展示
- 四、 水平标定线角度检测功能 (LineAngle)
- 4.1 业务场景与技术原理
- 4.2 核心接口说明 (Wafer_FindHorizontalLineAngle 单视野检测)
- 4.3 C# 调用代码示例
- 4.4 单视野实测效果展示
- 4.5 跨双视野大基线全局角度检测 (Wafer_FindTwoViewHorizontalLineAngle)
- 4.6 双视野综合诊断看板大图 (3000 × 1800)
- 五、 镜头畸变校正功能 (DistortionCorrection)
- 5.1 业务场景与技术架构
- 5.2 核心数据结构：WaferDotGridDistortionTemplate

- 5.3 业务调用流程与设计逻辑 (离线标定 vs 在线生产)
- 5.4 接口定义与参数详解
  - 1. Wafer_CreateDotGridDistortionTemplate (单视野标定)
  - 2. Wafer_CreateMultiViewDotGridDistortionTemplate (5视野联合标定)
  - 3. Wafer_SaveDistortionTemplateToFile (标定配方持久化保存)
  - 4. Wafer_LoadDistortionTemplateFromFile (标定配方持久化载入)
  - 5. Wafer_CorrectImageByDotGridTemplate (图像实时高速畸变校正)
- 5.5 C# 完整端到端调用示例
- 5.6 实测标定与校正效果展示
- 六、 晶圆圆中心与图像中心差值功能 (CircleCenterOffset)
- 6.1 业务场景与技术原理
- 6.2 核心接口说明 (Wafer_FindCircleCenterOffset)
- 6.3 C# 调用代码示例
- 6.4 实测效果展示
- 七、 平台旋转中心计算功能 (RotationCenter)
- 7.1 业务场景与旋转采样点数工程选型
- 7.2 核心数据结构：WaferPlatformRotationCenterResult
- 7.3 核心接口说明 (Wafer_CalculatePlatformRotationCenter)
- 7.4 C# 调用代码示例
- 7.5 实测拟合大结果图展示
- 八、 反射率光电响应标定与灰度线性化 LUT 功能 (ReflectanceLut)
- 8.1 业务场景与物理基准原理
- 8.2 核心数据结构：WaferReflectanceLutConfig 与 WaferReflectanceLutResult
- 8.3 核心接口说明 (Wafer_CalibrateReflectanceLut 与 Wafer_ApplyLutToImage)
- 8.4 C# 调用代码示例
- 8.5 实测标定与综合诊断大图展示
- 九、 9点标定与在线坐标转换功能 (AxisPixelCalibration)
- 9.1 业务场景与走位轨迹拓扑推导
- 9.2 业务调用流程与极简设计逻辑 (无句柄透明缓存)
- 9.3 核心接口说明 (极简 3 接口参数详解)

- 9.4 C# 调用代码示例 (纯内存流与在线对位)
- 9.5 实测标定与综合诊断大图展示
- 十、 四视野晶圆外缘圆心功能 (FourViewEdgeCircle)
- 10.1 采集条件与坐标约定
- 10.2 结果结构体与六参数接口
- 10.3 C# 调用示例
- 10.4 自适应边缘定位与错误处理
- 10.5 两组图片 DLL 实测结果与精度边界
- 十一、 八视野晶圆外缘圆心与 notch 角度功能 (EightViewEdgeCircle)
- 11.1 八方向采集顺序与坐标约定
- 11.2 结果结构体与六参数接口
- 11.3 C# 调用示例
- 11.4 notch 定位与错误处理
- 11.5 DLL 与 C# 实测结果及标定限制
- 11.6 从四图迁移到八图
- 十二、 平场校正与暗场-平场-LUT极速预处理流水线 (FlatField & PreprocessPipeline)
- 12.1 业务场景与物理链路架构
- 12.2 离线标定流程与物理因果逻辑
- 12.3 在线流水线处理架构：分步串行 vs 融合单次访存
- 12.4 核心数据结构与极简接口参数详解
  - 1. Wafer_CalibrateFlatField (平场标定接口)
  - 2. Wafer_ApplyFlatFieldCorrection (实时平场校正接口)
  - 3. Wafer_ApplyPreprocessPipeline (三合一极速预处理流水线接口)
- 12.5 C# P/Invoke 原生互操作与端到端示例
- 12.6 32位浮点 TIFF 配方存储与指针零损传输规范
- 12.7 真实产线实测性能与效果验证
- 附录：C++ 原生模块架构参考

一、 通用约定与基础规范

1.1 图像格式与内存对齐

- **输入图像格式**：全算法模块均针对工业灰度相机设计，输入数据为标准 **Mono8**（灰度图，单通道 8 位，`CV_8UC1`），每个像素占用 1 字节。
- **图像行对齐与步长**：图像内存必须为连续紧凑存储，行步长（Step / Stride）固定等于图像宽度 `width`。
- **缓冲区大小分配**：
- Mono8 输入/输出图像：缓冲区最小长度为 `width * height` 字节。
- 普通单图诊断/标注彩色结果图（`result_bgr`）：缓冲区最小长度为 `width * height * 3` 字节，像素排列遵循标准 **BGR** 顺序（每个像素依次为 Blue、Green、Red）。
- **5视野联合标定诊断大图（`result_bgr`）**：因输出为 **3*3 空间拓扑无损拼接大图**，输出分辨率为 `(width * 3) * (height * 3)`，所需缓冲区最小长度为 **`(width * 3) * (height * 3) * 3` 字节。**
- **内存生命周期原则**：所有由接口传出的图像缓冲区、模板结构体均由**调用方（上层应用）预先分配与释放。**SDK 内部绝不在堆上为输出图像重新申请内存，杜绝内存泄漏隐患。

1.2 SDK 状态返回码

所有 C API 函数的整型返回值均表示执行状态：

| 错误码常量 | 数值 | 含义说明 | 上层应用排查与处理建议 |

|---|---:|---|---|

| `WAFER_SUCCESS` | 0 | 执行成功 | 可正常读取输出数据或输出图像。 |

| `WAFER_ERR_INVALID_PARAM` | 1 | 参数无效 | 检查是否传入了空指针（`IntPtr.Zero`）、负数宽高、不支持的算法模式或超出合法范围的参数。 |

| `WAFER_ERR_IMAGE_EMPTY` | 2 | 图像为空 | 检查图像指针是否为 NULL 或长度为 0。 |

| `WAFER_ERR_FORMAT_MISMATCH` | 3 | 尺寸或格式不匹配 | 确保校正图像尺寸与畸变模板中的 `image_width` / `image_height` 严格相等。 |

| `WAFER_ERR_FEATURE_NOT_FOUND` | 4 | 未识别到特征 | 标定板未入画、曝光过暗/过曝、对比度不足，或特征圆/十字被遮挡。 |

| `WAFER_ERR_FITTING_FAILED` | 5 | 几何数学拟合失败 | 有效特征点过少（少于 10 个），导致多项式或直线最小二乘求解奇异。 |

| `WAFER_ERR_FILE_IO` | 6 | 文件读写失败 | 检查目标路径是否存在、是否具备写入权限或配方文件是否已损坏。 |

| `WAFER_ERR_UNKNOWN` | 999 | 未分类异常 | 内部未捕获的严重异常，请检查底层运行日志。 |

> **调用准则**：在读取输出参数（如坐标、角度、校正图）之前，**必须首先断言返回值等于 0（`WAFER_SUCCESS`）。**八视野接口另有部分结果约定：返回 0 表示圆心有效，角度仍须检查是否为有限数值，详见 11.4 节。

1.3 C# P/Invoke 原生互操作基类定义

在 C# 客户端项目中，建议统一封装一个名为 `WaferCalibNative.cs` 的静态互操作类（已剔除任何基于文件路径的接口，纯内存流交互）：

```
using System;
using System.Runtime.InteropServices;

namespace WaferVisionSystem.Interop {
    /// <summary>
    /// WaferCalibSDK 原生 C API P/Invoke 声明类
    /// </summary>
    public static class WaferCalibNative {
        public const string DllName = "WaferCalibSDK.dll";

        // 状态码常量
        public const int WAFER_SUCCESS = 0;
        public const int WAFER_ERR_INVALID_PARAM = 1;
        public const int WAFER_ERR_IMAGE_EMPTY = 2;
        public const int WAFER_ERR_FORMAT_MISMATCH = 3;
        public const int WAFER_ERR_FEATURE_NOT_FOUND = 4;
        public const int WAFER_ERR_FITTING_FAILED = 5;
        public const int WAFER_ERR_FILE_IO = 6;
        public const int WAFER_ERR_UNKNOWN = 999;

        // 畸变多项式映射系数数量 (三阶 2D 多项式固定为 10)
        public const int PolynomialCoefficientCount = 10;

        /// <summary>
        /// 畸变校正模板结构体 (与 C 结构体 WaferDotGridDistortionTemplate 顺序和字节严格对齐)
        /// </summary>
        [StructLayout(LayoutKind.Sequential)]
        public struct WaferDotGridDistortionTemplate {
```

```
    public int ImageWidth;
    public int ImageHeight;

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = PolynomialCoefficientCount)]
    public double[] OutputToInputX;

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = PolynomialCoefficientCount)]
    public double[] OutputToInputY;

    public int DetectedPointCount;
    public double RmsErrorPixels;
}

// ================= 各业务功能接口 P/Invoke 声明 =================

// 1. 暗场背景降噪
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_CreateDarkFrameTemplate(
    IntPtr[] frameBuffers,
    int frameCount,
    IntPtr outDarkTemplate,
    int width,
    int height,
    int method
);

// 2. 四十字 Mark 对角线交点定位
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_FindFourCrossMarkCenter(
    IntPtr imageBuffer,
    int width,
    int height,
```

```
    out double calculatedCenterX,
    out double calculatedCenterY,
    IntPtr resultBgr
);

// 3.1 水平标定线单视野角度检测
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_FindHorizontalLineAngle(
    IntPtr imageBuffer,
    int width,
    int height,
    out double angleDegrees,
    IntPtr resultBgr
);

// 3.2 跨双视野大基线高精度水平标定线全局角度检测
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_FindTwoViewHorizontalLineAngle(
    IntPtr mono8View1,
    IntPtr mono8View2,
    int width,
    int height,
    double stageDeltaXMm,
    double pixelScaleYUm,
    out double outGlobalAngleDeg,
    out double outView1AngleDeg,
    out double outView2AngleDeg,
    IntPtr diagnosticBgr,
    int diagWidth,
    int diagHeight
);
```

```
// 4.1 畸变校正 - 单视野点阵标定
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_CreateDotGridDistortionTemplate(
    IntPtr calibrationImageBuffer,
    int width,
    int height,
    int gridColumns,
    int gridRows,
    double pointSpacingMm,
    ref WaferDotGridDistortionTemplate outTemplate,
    IntPtr resultBgr
);

// 4.2 畸变校正 - 5视野联合多点位点阵标定
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_CreateMultiViewDotGridDistortionTemplate(
    IntPtr[] imageBuffers,
    int width,
    int height,
    int gridColumns,
    int gridRows,
    double pointSpacingMm,
    double stageStepMm,
    ref WaferDotGridDistortionTemplate outTemplate,
    IntPtr resultBgr
);

// 4.3 畸变校正 - 标定模板持久化保存到文件
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]
public static extern int Wafer_SaveDistortionTemplateToFile(
    string filePath,
```

```
    ref WaferDotGridDistortionTemplate distortionTemplate
);

// 4.4 畸变校正 - 从文件加载已标定的模板
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]
public static extern int Wafer_LoadDistortionTemplateFromFile(
    string filePath,
    ref WaferDotGridDistortionTemplate outTemplate
);

// 4.5 畸变校正 - 图像高速畸变校正 (生产在线)
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_CorrectImageByDotGridTemplate(
    IntPtr imageBuffer,
    int width,
    int height,
    ref WaferDotGridDistortionTemplate distortionTemplate,
    IntPtr correctedMono8
);

// 5. 晶圆圆中心与图像中心差值
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_FindCircleCenterOffset(
    IntPtr imageBuffer,
    int width,
    int height,
    out double offsetX,
    out double offsetY,
    IntPtr resultBgr
);
```

```
// 四视野晶圆外缘圆心：五个double，8字节对齐，总大小40字节
[StructLayout(LayoutKind.Sequential, Pack = 8)]
public struct WaferEdgeCircleResult {
    public double CenterXmm;
    public double CenterYmm;
    public double RadiusMm;
    public double RmsResidualUm;
    public double MaxResidualUm;
}

[DllImport(DllName, ExactSpelling = true,
    CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_FindFourViewEdgeCircle(
    [In, MarshalAs(UnmanagedType.LPArray, SizeConst = 4)]
    IntPtr[] imageBuffers,
    [In, MarshalAs(UnmanagedType.LPArray, SizeConst = 8)]
    double[] stageXYmm,
    int width,
    int height,
    double pixelSizeUm,
    out WaferEdgeCircleResult result
);

// 八视野晶圆外缘圆心与notch角度：六个double，共48字节
[StructLayout(LayoutKind.Sequential, Pack = 8)]
public struct WaferEightViewEdgeCircleResult {
    public double CenterXmm;
    public double CenterYmm;
    public double RadiusMm;
    public double RmsResidualUm;
    public double MaxResidualUm;
    public double NotchAngleDeg;
```

```
}

[DllImport(DllName, ExactSpelling = true,
    CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_FindEightViewEdgeCircle(
    [In, MarshalAs(UnmanagedType.LPArray, SizeConst = 8)]
    IntPtr[] imageBuffers,
    [In, MarshalAs(UnmanagedType.LPArray, SizeConst = 16)]
    double[] stageXYmm,
    int width,
    int height,
    double pixelSizeUm,
    out WaferEightViewEdgeCircleResult result
);

// 平台旋转中心最大支持点数
public const int MaxRotationCenterPoints = 36;

/// <summary>
/// 平台旋转中心计算结果结构体 (与 C 结构体 WaferPlatformRotationCenterResult 内存对齐)
/// </summary>
[StructLayout(LayoutKind.Sequential)]
public struct WaferPlatformRotationCenterResult {
    public double CenterX;
    public double CenterY;
    public double Radius;
    public double RmsError;
    public double MaxError;
    public int ValidPointCount;

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = MaxRotationCenterPoints)]
```

```
    public double[] PointX;

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = MaxRotationCenterPoints)]
    public double[] PointY;

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = MaxRotationCenterPoints)]
    public double[] PointResidual;
}

// 6. 平台旋转中心计算
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_CalculatePlatformRotationCenter(
    IntPtr[] imageBuffers,
    int imageCount,
    int width,
    int height,
    ref WaferPlatformRotationCenterResult outResult,
    IntPtr resultBgr
);

/// <summary>
/// 反射率光电响应标定配置结构体
/// </summary>
[StructLayout(LayoutKind.Sequential)]
public struct WaferReflectanceLutConfig {
    public int LutMode;              // 0: 分段阶梯死区模式; 1: 单调平滑样条模式(PCHIP)
    public int RoiCenterWidth;       // 中心测量 ROI 宽度 (像素)，0 表示全幅
    public int RoiCenterHeight;      // 中心测量 ROI 高度 (像素)，0 表示全幅
    public int DeadbandWidth;        // 死区半宽 (像素灰度级，默认 5)

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 4)]
    public double[] TargetValues;    // 目标物理基准灰度 [13, 128, 192, 230]
```

```
}

/// <summary>
/// 反射率光电响应标定结果结构体
/// </summary>
[StructLayout(LayoutKind.Sequential)]
public struct WaferReflectanceLutResult {
    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 256)]
    public byte[] Lut;               // 256 元素灰度映射查找表

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 4)]
    public double[] MeasuredPeaks;   // 实测波峰灰度值

    [MarshalAs(UnmanagedType.ByValArray, SizeConst = 4)]
    public double[] TargetValues;    // 标定使用的目标基准值

    public double RawLinearityR2;    // 校正前非线性 R^2
    public double CorrectedLinearityR2; // 校正后线性 R^2
}

// 7. 反射率光电响应标定与 LUT 生成 (离线标定)
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_CalibrateReflectanceLut(
    IntPtr[] imageBuffers,
    int width,
    int height,
    ref WaferReflectanceLutConfig config,
    ref WaferReflectanceLutResult outResult,
    IntPtr resultBgr
);
```

```
// 8. 单帧图像高速查表线性化校正 (在线生产, 每帧 ~0.5ms)
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_ApplyLutToImage(
    IntPtr srcMono8,
    int width,
    int height,
    byte[] lut256,
    IntPtr dstMono8
);

// 9. LUT 配方文件保存与载入
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]
public static extern int Wafer_SaveLutToFile(string filePath, byte[] lut256);

[DllImport(DllName, CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]
public static extern int Wafer_LoadLutFromFile(string filePath, byte[] lut256);

// 10. 9点标定计算 (极简接口：传入初始坐标与步长，直接输出 X/Y 像元物理当量，自动导出配方 JSON 与诊断大图)
[DllImport(DllName, CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]
public static extern int Wafer_CalibrateAxisPixelGrid(
    IntPtr[] mono8Buffers,
    int imageCount,
    int width,
    int height,
    double initialAxisX,
    double initialAxisY,
    double stepSizeMm,
    string outputCalibJsonPath,
    out double outPixelScaleXUm,
    out double outPixelScaleYUm,
    IntPtr diagnosticBgr,
    int diagWidth,
```

```
            int diagHeight
        );

        // 11. 在线高速坐标正变换 (像素 -> 轴，底层基于修改时间戳透明内存缓存，无需管理句柄)
        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]
        public static extern int Wafer_TransformPixelToAxis(
            string calibJsonPath,
            double u,
            double v,
            out double outAxisX,
            out double outAxisY
        );

        // 12. 在线高速坐标逆变换 (轴 -> 像素，底层基于修改时间戳透明内存缓存，无需管理句柄)
        [DllImport(DllName, CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]
        public static extern int Wafer_TransformAxisToPixel(
            string calibJsonPath,
            double axisX,
            double axisY,
            out double outU,
            out double outV
        );
    }
}
```

1.4 工业相机采集内存传递规范

在主流工业相机 SDK（如海康 MVS、大恒 Galaxy、巴斯勒 Pylon 等）的回调函数中，相机的原始数据指针为非托管指针 `IntPtr`。

- **零拷贝直传（推荐）**：可直接将相机回调提供的 `pFrameInfo.pBufAddr` 或 `IntPtr` 传入 `WaferCalibSDK`，无需通过 `Marshal.Copy` 复制到托管 `byte[]` 中，最大限度节省 CPU 内存带宽与 GC 压力。
- **托管数组传递**：若图像已在 C# `byte[]` 数组中，可使用 `GCHandle.Alloc(bytes, GCHandleType.Pinned)` 固定句柄后获取其指针，调用结束后必须调用 `handle.Free()` 释放。

二、 暗场背景降噪功能 (NoiseReduction)

2.1 业务场景与技术原理

在半导体晶圆微观缺陷检测与高精度对准中，CIS/CCD 图像传感器受热噪声、暗电流不均匀性及暗信号非均匀性（DSNU/FPN）影响，在无光照射或微光背景下存在固定底噪，降低弱边缘对比度。

该功能将相机在遮光闭光环境下连续采集的 N 帧暗场图像进行统计学合成，建立高信噪比的背景降噪基准模板（Dark Frame Template）。

2.2 核心接口说明 (Wafer_CreateDarkFrameTemplate)

```
int Wafer_CreateDarkFrameTemplate(
    const unsigned char** frame_buffers,
    int frame_count,
    unsigned char* out_dark_template,
    int width,
    int height,
    int method
);
```

参数详解

参数名	传递方向	数据类型	说明
`frame_buffers`	输入	`const unsigned char**`	包含 `frame_count` 个指向 Mono8 图像首地址的指针数组。每帧内存大小为 `width * height` 字节。
`frame_count`	输入	`int`	参与合成的暗场图像帧数，通常建议 8~32 帧以获得最佳信噪比。
`out_dark_template`	输出	`unsigned char*`	调用方预先分配的输出模板缓冲区，大小为 `width * height` 字节。
`width`	输入	`int`	图像宽度（像素）。
`height`	输入	`int`	图像高度（像素）。
`method`	输入	`int`	合成算法策略： • `0`：**平均值法 (Mean)**，平滑高斯白噪声最快； • `1`：**中值滤波法 (Median)**，强力剔除传感器随机坏点和脉冲散粒噪声； • `2`：**3-Sigma 剔除均值法 (Truncated Mean)**，半导体级最高精度合成。

2.3 C# 调用代码示例

```
using System;

using System.Runtime.InteropServices;

using WaferVisionSystem.Interop;


public class NoiseReductionDemo {

    public static void GenerateDarkTemplate(IntPtr[] cameraDarkFrames, int width, int height) {

        int frameCount = cameraDarkFrames.Length;

        int bufferSize = width * height;

        IntPtr outTemplatePtr = Marshal.AllocHGlobal(bufferSize);


        try {

            // 使用 3-Sigma 剔除均值法 (method = 2) 合成高质量暗场底模

            int ret = WaferCalibNative.Wafer_CreateDarkFrameTemplate(

                cameraDarkFrames,

                frameCount,

                outTemplatePtr,

                width,

                height,

                2

            );


            if (ret != WaferCalibNative.WAFER_SUCCESS) {

                throw new Exception($"暗场模板合成失败，错误码: {ret}");

            }


            // 读取合成出的 Mono8 模板数据 (可持久化保存为标定底图)

            byte[] darkTemplateBytes = new byte[bufferSize];

            Marshal.Copy(outTemplatePtr, darkTemplateBytes, 0, bufferSize);

            Console.WriteLine("暗场模板生成成功! ");

        } finally {
```

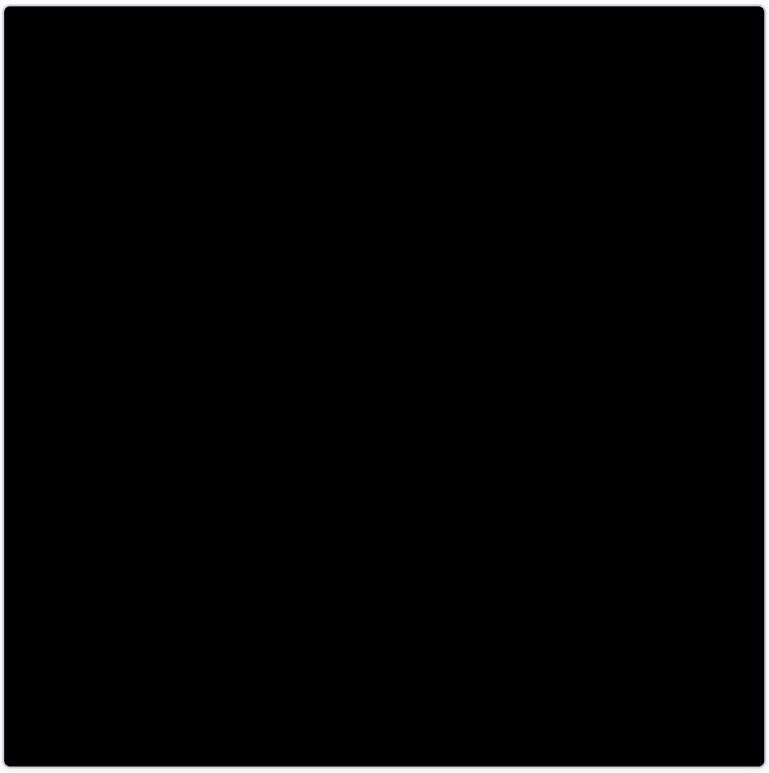
```
            Marshal.FreeHGlobal(outTemplatePtr);

        }

    }

}
```

2.4 实测效果展示

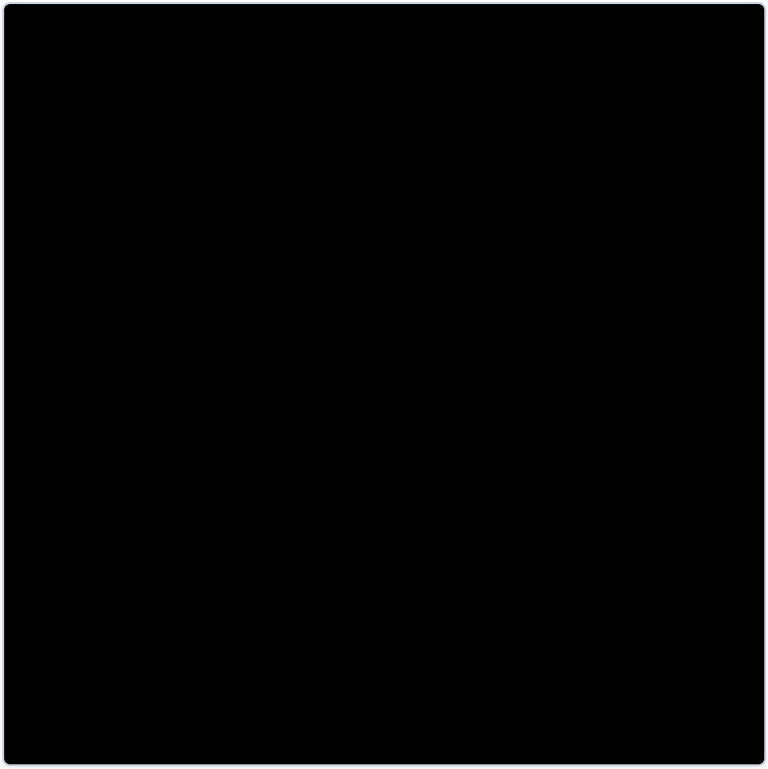
| 输入单帧暗场原始图像 | 输出统计合成降噪模板 |

|:---:|:---:|

|



|



|

| *可见传感器固有固定模式噪声与暗电流斑驳条纹* | *通过多帧 3-Sigma 剔除平滑后的干净基准底模* |

三、 四十字 Mark 对角线交点定位功能 (MarkCenter)

3.1 业务场景与技术原理

在晶圆切割、划片、光刻与粗对准工步中，载片台四角或特定视场布置有四个高反差十字 Mark。为了建立晶圆全局坐标基准，需定位该四个十字 Mark 并计算其两组空间对角线的几何中心交点。

算法流程包含：

1. 自适应阈值分割与十字几何拓扑筛选；
2. 亚像素轮廓边缘提取与角点重心亚像素精定位；
3. 空间拓扑分配：自动识别左上、右上、左下、右下四个 Mark；
4. 求解对角线 $L_1(P_{TL}, P_{BR})$ 与 $L_2(P_{TR}, P_{BL})$ 的解析几何交点 (C_x, C_y)。

3.2 核心接口说明 (Wafer_FindFourCrossMarkCenter)

```
int Wafer_FindFourCrossMarkCenter(

    const unsigned char* image_buffer,

    int width,

    int height,

    double* calculated_center_x,
```

```
    double* calculated_center_y,
    unsigned char* result_bgr
);
```

参数详解

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `image_buffer` | 输入 | `const unsigned char*` | 输入 Mono8 图像指针，长度为 `width * height` 字节。 |

| `width` | 输入 | `int` | 图像宽度（像素）。 |

| `height` | 输入 | `int` | 图像高度（像素）。 |

| `calculated_center_x` | 输出 | `double*` | 两对角线交点的亚像素 X 坐标（以图像左上角为原点）。 |

| `calculated_center_y` | 输出 | `double*` | 两对角线交点的亚像素 Y 坐标（以图像左上角为原点）。 |

| `result_bgr` | 输出 | `unsigned char*` | 输出 BGR 彩色标注图缓冲区（大小为 `width * height * 3` 字节），包含十字标记框、对角线与中心交点十字标注。传 NULL 表示不输出。 |

3.3 C# 调用代码示例

```
using System;
using System.Runtime.InteropServices;
using WaferVisionSystem.Interop;


public class MarkCenterDemo {
    public static void LocateWaferMarkCenter(IntPtr imageBuffer, int width, int height) {
        int bgrSize = width * height * 3;
        IntPtr resultBgrPtr = Marshal.AllocHGlobal(bgrSize);


        try {
            int ret = WaferCalibNative.Wafer_FindFourCrossMarkCenter(
                imageBuffer,
                width,
                height,
                out double centerX,
```

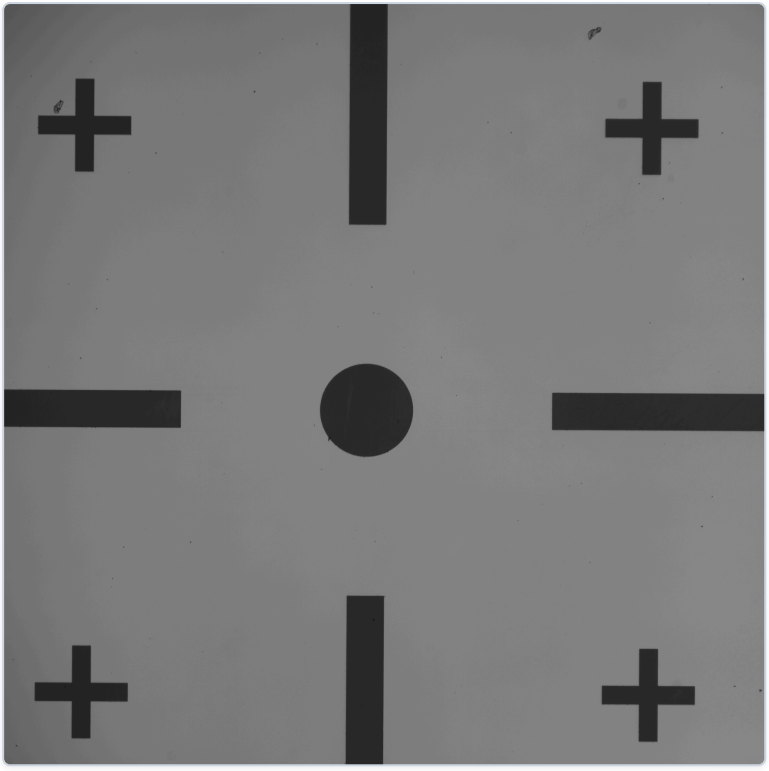
```
                out double centerY,
                resultBgrPtr
            );

            if (ret != WaferCalibNative.WAFER_SUCCESS) {
                throw new Exception($"十字 Mark 定位失败，错误码: {ret}");
            }

            Console.WriteLine($"四十字 Mark 对角线交点亚像素坐标: X = {centerX:F3}, Y = {centerY:F3}");
            // 可将 resultBgrPtr 的 BGR 数据送入 UI 控件进行画面实时渲染呈现
        } finally {
            Marshal.FreeHGlobal(resultBgrPtr);
        }
    }
}
```

3.4 实测效果展示

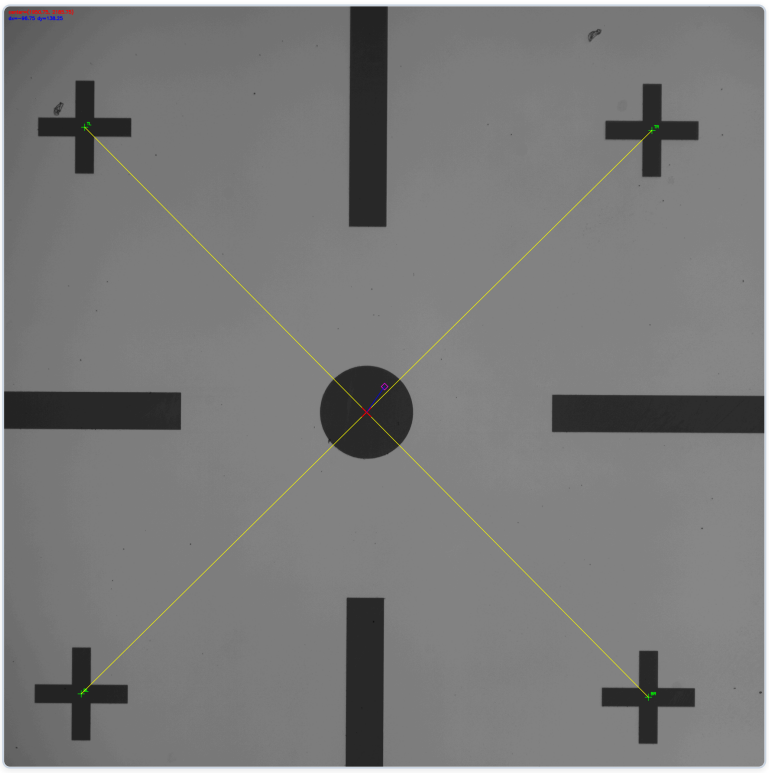
| 输入晶圆四十字 Mark 图像 | 输出亚像素标注与对角线交点诊断图 |

|:---:|:---:|

|



|



|

| *含有 4 个高对比度十字标记点的晶圆微观原图* | *精确定位 4 个十字亚像素中心，绘制对角线并求出交点坐标* |

四、 水平标定线角度检测功能 (LineAngle)

4.1 业务场景与技术原理

半导体晶圆通常具备平边（Flat）或基准直边，机械手臂在抓取放置后存在微小旋转安装偏角 θ。为了实现晶圆角度自动对正，需要高精度测量该参考直线的水平倾角。

算法通过高精度梯度边缘检测子采样提取直线上数百个亚像素边缘点，结合 RANSAC 鲁棒直线回归拟合算法，输出物理角度值 θ（角度精度优于 $\pm 0.005^\circ$）。

角度坐标系定义：

- 范围：$[-90.0^\circ, 90.0^\circ)$；
- 水平基准：完美水平线角度为 0.0°；
- 符号规则：直线从左至右**向下倾斜为正角度 ($+$)**，从左至右**向上翘起为负角度 ($-$)**。

4.2 核心接口说明 (Wafer_FindHorizontalLineAngle)

```
int Wafer_FindHorizontalLineAngle(
    const unsigned char* image_buffer,
    int width,
    int height,
    double* angle_degrees,
    unsigned char* result_bgr
);
```

参数详解

参数名	传递方向	数据类型	说明
`image_buffer`	输入	`const unsigned char*`	输入 Mono8 图像指针，长度为 `width * height` 字节。
`width`	输入	`int`	图像宽度（像素）。
`height`	输入	`int`	图像高度（像素）。
`angle_degrees`	输出	`double*`	检测到的直线水平偏角，单位为度（Degree）。
`result_bgr`	输出	`unsigned char*`	输出 BGR 彩色标注图缓冲区（`width * height * 3` 字节），包含采样边缘点集与拟合参考线。传 NULL 表示不输出。

4.3 C# 调用代码示例

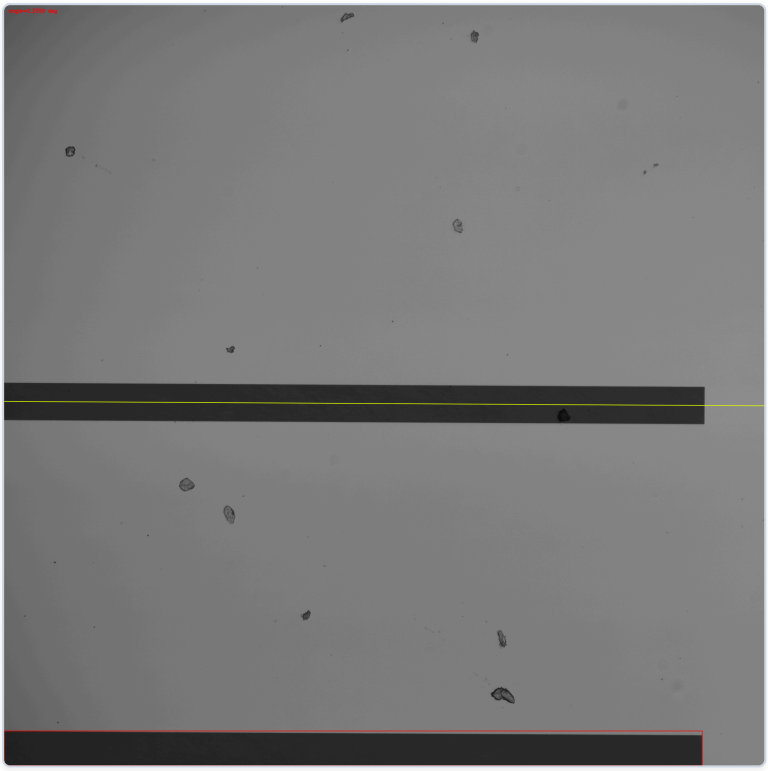
```
using System;

using System.Runtime.InteropServices;

using WaferVisionSystem.Interop;


public class LineAngleDemo {

    public static double MeasureWaferRotation(IntPtr imageBuffer, int width, int height) {

        int bgrSize = width * height * 3;

        IntPtr resultBgrPtr = Marshal.AllocHGlobal(bgrSize);


        try {

            int ret = WaferCalibNative.Wafer_FindHorizontalLineAngle(

                imageBuffer,

                width,

                height,

                out double angleDeg,

                resultBgrPtr

            );


            if (ret != WaferCalibNative.WAFER_SUCCESS) {

                throw new Exception($"水平线角度检测失败，错误码: {ret}");

            }


            Console.WriteLine($"检测到晶圆基准线倾角: {angleDeg:F4}° (旋转平台需反向补偿 {-angleDeg:F4}°)");

            return angleDeg;

        } finally {

            Marshal.FreeHGlobal(resultBgrPtr);

        }

    }

}
```

4.4 单视野实测效果展示

| 水平基准线提取与高精度角度检测诊断图 |

|:---:|

|



|

| *绿线为 RANSAC 鲁棒拟合直线，实时输出极小微米级倾角偏差* |

4.5 跨双视野大基线全局角度检测 (Wafer_FindTwoViewHorizontalLineAngle)

4.5.1 业务场景与技术原理

在超高精度的半导体晶圆校平与旋转粗/精对位工序中，若仅使用单个局部视场（如 4096 像素，在约 $0.9009\ \mu\mathrm{m/px}$ 像元下视场仅约 3.69 mm）来估算晶圆标定线角度，单像素采样跳动或边缘粗糙度会对角度带来约 $\pm 0.01^\circ \sim 0.03^\circ$ 的不确定度。

为了突破单视场的光学尺寸物理局限，本接口支持**大基线跨视场对准测角**：

1. **工作模式**：机台相机固定，机械载物台沿 X 轴方向平移大跨距物理行程（例如 $\Delta X_{\text{stage}} = 285.000\ \text{mm}$），在标定板或晶圆上的同一条水平线上分别拍摄最左端起点【视野 1】与最右端终点【视野 2】；
2. **机台约束**：由于机台在此过程中 Y 轴保持静止锁定，因此**无需传入任何 Y 轴位移参数**；
3. **几何解算**：
4. 算法在视野 1 提取亚像素直线，求出其在图像中心列 $u_c = W/2$ 处的精确像面高度 v_1；
5. 算法在视野 2 提取亚像素直线，求出其在图像中心列 $u_c = W/2$ 处的精确像面高度 v_2；
6. 两视野中心测点之间的像面垂直落差为 $\Delta v = v_2 - v_1$（像素）；

7. 结合垂直方向像元当量 $s_y = \text{pixel_scale_y_um}/1000.0$（mm/px，可直接取自 9 点标定输出参数），换算得到物理世界垂直落差：

$$\Delta Y_{\text{world}} = -(v_2 - v_1) \times \left(\frac{\text{pixel_scale_y_um}}{1000.0} \right) \quad (\text{mm})$$

8. 结合机械 X 轴位移 ΔX_{stage}（mm），解算超长基线下的全局绝对倾角：

$$\Theta_{\text{global}} = \arctan\left(\frac{\Delta Y_{\text{world}}}{\Delta X_{\text{stage}}} \right) \times \frac{180}{\pi} \quad (\text{度})$$

$$\Theta_{\text{arcmin}} = \Theta_{\text{global}} \times 60 \quad (\text{角分})$$

9. **精度增益**：

基线跨度由单图视场 3.69 mm 扩展至 285 mm，**几何测角信噪比与抗像元离散误差能力提升了约 77 倍**！测角精度直接进入角秒/超微米级。

4.5.2 核心接口说明 (Wafer_FindTwoViewHorizontalLineAngle)

```
int Wafer_FindTwoViewHorizontalLineAngle(
    const unsigned char* mono8_view1,
    const unsigned char* mono8_view2,
    int width,
    int height,
    double stage_delta_x_mm,
    double pixel_scale_y_um,
    double* out_global_angle_deg,
    double* out_view1_angle_deg,
    double* out_view2_angle_deg,
    unsigned char* diagnostic_bgr,
    int diag_width,
    int diag_height
);
```

参数详解

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `mono8_view1` | 输入 | `const unsigned char*` | 视野 1 (最左端起点) Mono8 图像指针，长度为 `width * height` 字节。 |

| `mono8_view2` | 输入 | `const unsigned char*` | 视野 2 (最右端终点) Mono8 图像指针，长度为 `width * height` 字节。 |

| `width` | 输入 | `int` | 图像宽度（像素，如 4096）。 |

| `height` | 输入 | `int` | 图像高度（像素，如 4096）。 |

| `stage_delta_x_mm` | 输入 | `double` | 机械 X 轴从视野 1 平移至视野 2 的物理位移绝对跨度（mm，如 285.000）。 |

| `pixel_scale_y_um` | 输入 | `double` | 像元垂直方向物理当量（微米/像素，如 0.9009，可直接由 9 点标定接口输出）。 |

| `out_global_angle_deg` | 输出 | `double*` | 输出：跨双视野大基线超高精度全局角度（度，范围 [-90, 90)）。 |

| `out_view1_angle_deg` | 输出 | `double*` | 可选输出：视野 1 单图局部拟合角度（度）；传 NULL 则忽略。 |

| `out_view2_angle_deg` | 输出 | `double*` | 可选输出：视野 2 单图局部拟合角度（度）；传 NULL 则忽略。 |

| `diagnostic_bgr` | 输出 | `unsigned char*` | 可选输出：双视野联合对齐综合诊断看板大图缓冲区（3 通道 BGR，大小为 `diag_width * diag_height * 3` 字节）；传 NULL 表示不生成，零耗时零开销。 |

| `diag_width` | 输入 | `int` | 诊断大图宽度（**推荐工业标准看板尺寸：3000**）。 |

| `diag_height` | 输入 | `int` | 诊断大图高度（**推荐工业标准看板尺寸：1800**）。 |

4.5.3 C# 调用代码示例

```
using System;

using System.Runtime.InteropServices;

using WaferVisionSystem.Interop;


public class TwoViewLineAngleDemo {

    public static void MeasureLongBaselineGlobalAngle(

        IntPtr view1Ptr, IntPtr view2Ptr, int width, int height) {


        // 工况参数：机械 X 轴移动 285 mm，像元 Y 当量 0.9009 um/px

        double stageDeltaXMm = 285.000;

        double pixelScaleYUm = 0.9009;


        // 诊断大图推荐尺寸：3000 x 1800 (BGR 3通道，约 16.2 MB)

        int diagWidth = 3000;

        int diagHeight = 1800;

        int diagBufferSize = diagWidth * diagHeight * 3;

        IntPtr diagBgrPtr = Marshal.AllocHGlobal(diagBufferSize);
```

```
try {
    int ret = WaferCalibNative.Wafer_FindTwoViewHorizontalLineAngle(
        view1Ptr,
        view2Ptr,
        width,
        height,
        stageDeltaXMm,
        pixelScaleYUm,
        out double globalAngleDeg,
        out double view1AngleDeg,
        out double view2AngleDeg,
        diagBgrPtr,
        diagWidth,
        diagHeight
    );

    if (ret != WaferCalibNative.WAFER_SUCCESS) {
        throw new Exception($"双视野大基线标定线测角失败，错误码: {ret}");
    }

    double globalArcmin = globalAngleDeg * 60.0;
    Console.WriteLine($"[大基线测角成功]");
    Console.WriteLine($"  全局大基线直线角度: {globalAngleDeg:F4}° ({globalArcmin:F2} 角分)");
    Console.WriteLine($"  左端视野1单图角度:  {view1AngleDeg:F4}°");
    Console.WriteLine($"  右端视野2单图角度:  {view2AngleDeg:F4}°");
    Console.WriteLine($"  两端局部一致性偏差: {Math.Abs(view1AngleDeg - view2AngleDeg):F4}°");

    // 可将 diagBgrPtr 写入位图或送入工控大屏 PictureBox 显示
} finally {
    Marshal.FreeHGlobal(diagBgrPtr);
```

```
        }
    }
}
```

4.6 双视野综合诊断看板大图 (3000 × 1800)

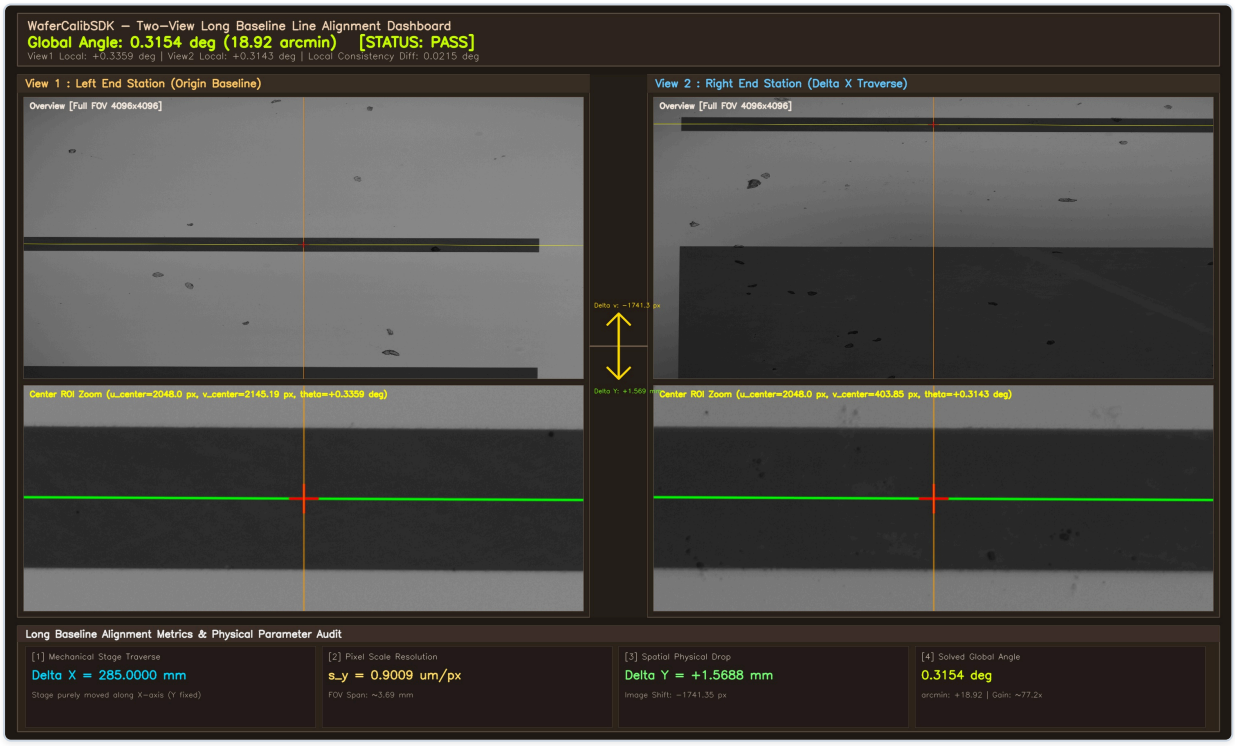
针对 4096×4096 高分辨率图像，直接并排（8192×4096）将产生高达 100MB 内存开销并引起工控界面渲染卡顿。本 SDK 精心打造了 `3000 × 1800` **工业级宽屏全景综合看板**：

- **顶部 HUD 状态栏**：醒目绿色大字呈现全局角度与角分值、通过状态、两端单视野局部角度与一致性偏差；
- **主体双视口对比区**：
- 左通道：展示视野 1 全景缩略图及直线走向 + 中心测点亚像素 1:1 ROI 特写图（标明 $v_1 = 2145.19$ px）；
- 右通道：展示视野 2 全景缩略图及直线走向 + 中心测点亚像素 1:1 ROI 特写图（标明 $v_2 = 403.85$ px）；
- 中间对齐桥：直观标尺与双向箭头展示垂直像面落差 $\Delta v = -1741.3$ px 与物理落差 $\Delta Y = +1.569$ mm；
- **底部指标卡片区**：详细列出机械行程、像元分辨率当量、空间落差、解算公式展开与 77 倍基线信噪比增益。

| 跨双视野大基线水平标定线联合对齐综合诊断看板大图 (3000 × 1800) |

|:---:|

|



|

| *实测 4096×4096 双视野解算：全局大基线角度 0.3154° (18.92 角分)，基线跨度 285mm，像面落差 -1741.35px* |

五、 镜头畸变校正功能 (DistortionCorrection)

5.1 业务场景与技术架构

高倍率远心镜头与显微镜头普遍存在枕形、桶形光学畸变，导致图像边缘尺寸拉伸或压缩，严重影响亚微米级量测精度。

本 SDK 提供基于高阶二维多项式的反畸变模型：

- **单视野标定 (`Wafer_CreateDotGridDistortionTemplate`)**：适用于标定板尺寸大于相机视野（标定板能充满整个图像像面）的标准工况。
- **5视野联合标定 (`Wafer_CreateMultiViewDotGridDistortionTemplate`)**：适用于相机视野较大、标定板尺寸无法覆盖全视野的微观场景。在相机位置固定的前提下，通过载有标定板的平台平移至**左上、左下、右上、右下、中心** 5 个点位进行拍摄，由算法自动识别白圆与特征圆、对齐外参并联合优化，生成全局单一校正模板。

5.2 核心数据结构：WaferDotGridDistortionTemplate

```
typedef struct WaferDotGridDistortionTemplate {

    int image_width;

    int image_height;

    double output_to_input_x[10];

    double output_to_input_y[10];

    int detected_point_count;

    double rms_error_pixels;

} WaferDotGridDistortionTemplate;
```

字段物理意义

| 结构体字段 | 数据类型 | 物理含义 | 在线生产校正时是否使用 |

|---|---|---|:---:|

| `image_width` | `int` | 标定时的图像宽度（像素）。用于安全性校验，防止模板被错用于不同分辨率相机。 | **是** |

| `image_height` | `int` | 标定时的图像高度（像素）。 | **是** |

| `output_to_input_x[10]` | `double[10]` | 二维三阶反向映射多项式在 X 方向的 10 个解算系数。 | **是** |

| `output_to_input_y[10]` | `double[10]` | 二维三阶反向映射多项式在 Y 方向的 10 个解算系数。 | **是** |

| `detected_point_count` | `int` | 参与标定拟合的总有效特征圆点数。 | 否 (仅供记录) |

| `rms_error_pixels` | `double` | 标定点阵的全局均方根拟合残差（RMS Error，单位像素），用于评定镜头标定质量（一般应小于 0.25 像素）。 | 否 (仅供记录) |

> **数学多项式映射模型**：

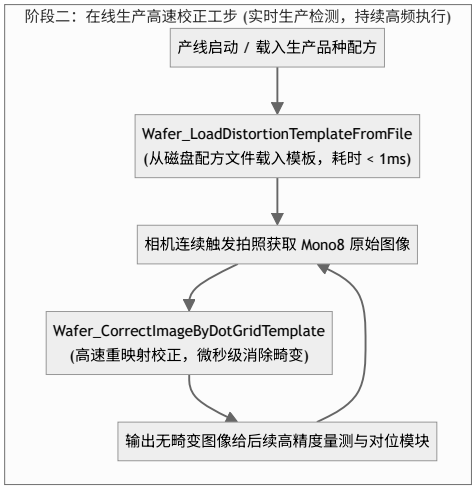
设校正后输出归一化坐标为 $(u, v) \in [0, 1]^2$，则原始输入畸变坐标 (x, y) 通过以下 10 项三阶多项式求得：

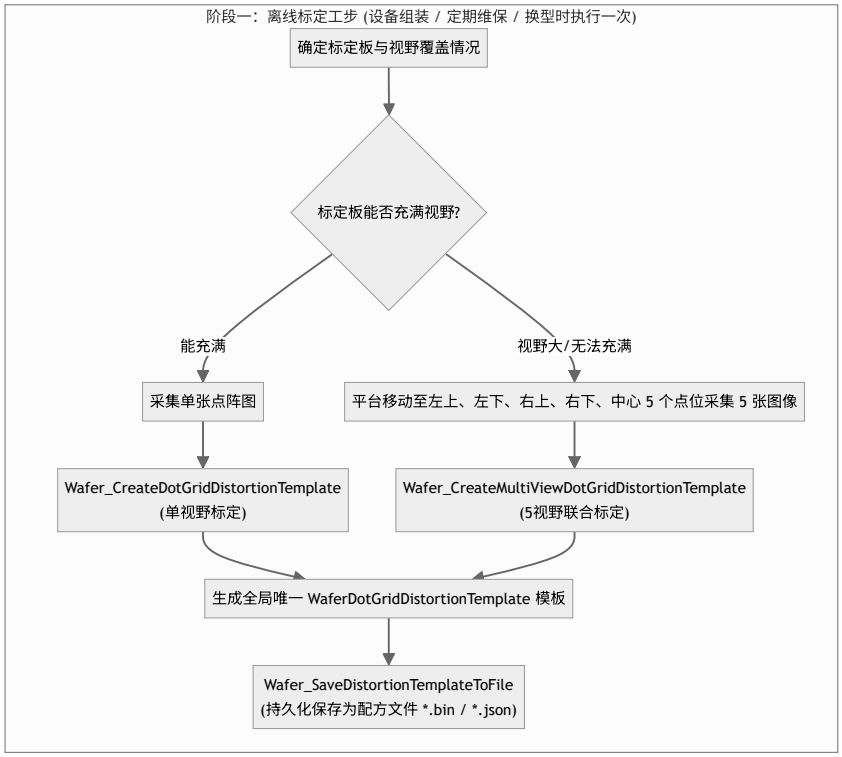
$$x = a_0 + a_1u + a_2v + a_3u^2 + a_4uv + a_5v^2 + a_6u^3 + a_7u^2v + a_8uv^2 + a_9v^3$$

$$y = b_0 + b_1u + b_2v + b_3u^2 + b_4uv + b_5v^2 + b_6u^3 + b_7u^2v + b_8uv^2 + b_9v^3$$

5.3 业务调用流程与设计逻辑 (离线标定 vs 在线生产)

工业产线视觉系统的工作流程严格划分为**离线标定阶段**与**在线高速生产校正阶段**：





5 视野联合标定算法逻辑拆解

1. **高信噪比亚像素圆心提取**：采用顶帽变换（Top-Hat）消除大视场光照不均匀，结合轮廓重心与椭圆拟合，亚像素提取图像中发白大圆；
2. **中心标记圆自动识别与拓扑索引**：自动定位标定板第 101 个中心特征圆，自适应恢复完整的 10×10 点阵行列物理网格索引；
3. **SVD 外参刚体初值估计**：利用视野重叠区域的对应特征点集，通过 SVD 奇异值分解自动解算各视野相对于中心视野的刚体变换初值；
4. **全局非线性联合优化**：构建全像面特征点集合，以最小化像面重投影残差为目标，求解全局单一多项式模型；
5. **无损全画幅 3x3 拼接诊断图**：将 5 视野原图按照物理移动拓扑无损拼接入 3x3 大图中，各自独立标注圆点与网格编号，直观展示全像面边缘覆盖度。

5.4 接口定义与参数详解

1. Wafer_CreateDotGridDistortionTemplate (单视野标定)

从单张规则圆点阵 Mono8 图像建立固定视野畸变校正模板。

```
int Wafer_CreateDotGridDistortionTemplate(
    const unsigned char* calibration_image_buffer,
    int width,
    int height,
    int grid_columns,
    int grid_rows,
    double point_spacing_mm,
    WaferDotGridDistortionTemplate* out_template,
    unsigned char* result_bgr
);
```

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `calibration_image_buffer` | 输入 | `const unsigned char*` | 标定点阵 Mono8 图像首地址，长度为 `width * height` 字节。 |

| `width` | 输入 | `int` | 图像宽度（像素）。 |

| `height` | 输入 | `int` | 图像高度（像素）。 |

| `grid_columns` | 输入 | `int` | 点阵网格列数（如 10）。 |

| `grid_rows` | 输入 | `int` | 点阵网格行数（如 10）。 |

| `point_spacing_mm` | 输入 | `double` | 相邻圆点的物理间距（单位：mm，例如 0.6 或 1.0）。 |

| `out_template` | 输出 | `WaferDotGridDistortionTemplate*` | 输出生成的畸变校正模板指针。 |

| `result_bgr` | 输出 | `unsigned char*` | 输出 BGR 诊断图缓冲区（`width * height * 3` 字节），传 NULL 表示不输出。 |

---

2. Wafer_CreateMultiViewDotGridDistortionTemplate (5视野联合标定)

结合 5 个视野图像缓冲区进行多视野联合优化标定，生成全画幅单一全局标定模板。

```
int Wafer_CreateMultiViewDotGridDistortionTemplate(
    const unsigned char** image_buffers,
    int width,
    int height,
```

```
    int grid_columns,
    int grid_rows,
    double point_spacing_mm,
    double stage_step_mm,
    WaferDotGridDistortionTemplate* out_template,
    unsigned char* result_bgr
);
```

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `image_buffers` | 输入 | `const unsigned char**` | 包含 5 个图像指针的数组，**顺序固定为：** `[中心，左上，左下，右上，右下]`。每个缓冲区大小为 `width * height` 字节。 |

| `width` | 输入 | `int` | 图像宽度（像素）。 |

| `height` | 输入 | `int` | 图像高度（像素）。 |

| `grid_columns` | 输入 | `int` | 标定板网格列数（通常为 10）。 |

| `grid_rows` | 输入 | `int` | 标定板网格行数（通常为 10）。 |

| `point_spacing_mm` | 输入 | `double` | 该倍率下标定板两圆物理间距（单位：mm，例如 1.5倍率为 1.0mm，5倍率为 0.6mm 等）。 |

| `stage_step_mm` | 输入 | `double` | 机台位移步长先验值（mm）。**若传 `0.0`，算法将启用全自动纯视觉拓扑自适应匹配。** |

| `out_template` | 输出 | `WaferDotGridDistortionTemplate*` | 输出计算得到的全局单一畸变模板指针。 |

| `result_bgr` | 输出 | `unsigned char*` | 输出 **3*3 空间拓扑无损拼接 BGR 诊断大图**缓冲区，输出分辨率为 `(width * 3) * (height * 3)`，所需缓冲区最小长度为 `(width * 3) * (height * 3) * 3` **字节**。中央为中心视野，四角对应机台 4 个边缘点位，各视野独立保留原图并标注自身圆点编号与残差，右侧集成全局标定汇总仪表盘。传 NULL 表示不输出。 |

---

3. Wafer_SaveDistortionTemplateToFile (标定配方持久化保存)

将计算得到的模板保存到磁盘配方文件中。

```
int Wafer_SaveDistortionTemplateToFile(
    const char* file_path,
    const WaferDotGridDistortionTemplate* distortion_template
);
```

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `file_path` | 输入 | `const char*` | 目标文件路径（支持 Windows 包含中文字符的路径）。推荐扩展名为 `.bin`（紧凑二进制格式）或 `.json`（人类可读文本格式）。 |

| `distortion_template` | 输入 | `const WaferDotGridDistortionTemplate*` | 待保存的标定模板指针。 |

---

4. Wafer_LoadDistortionTemplateFromFile (标定配方持久化载入)

从磁盘配方文件载入预先标定好的模板，耗时通常小于 1ms。

```
int Wafer_LoadDistortionTemplateFromFile(

    const char* file_path,

    WaferDotGridDistortionTemplate* out_template

);
```

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `file_path` | 输入 | `const char*` | 配方文件路径（支持 Windows 中文字符路径）。 |

| `out_template` | 输出 | `WaferDotGridDistortionTemplate*` | 接收读取数据的模板结构体指针。 |

---

5. Wafer_CorrectImageByDotGridTemplate (图像实时高速畸变校正)

使用预先加载好的模板对在线相机采集的 Mono8 图像进行高速畸变消除。

```
int Wafer_CorrectImageByDotGridTemplate(

    const unsigned char* image_buffer,

    int width,

    int height,

    const WaferDotGridDistortionTemplate* distortion_template,

    unsigned char* corrected_mono8

);
```

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `image_buffer` | 输入 | `const unsigned char*` | 原始待校正 Mono8 图像指针，长度为 `width * height` 字节。 |

| `width` | 输入 | `int` | 图像宽度（像素），**必须等于** `distortion_template->image_width`。 |

| `height` | 输入 | `int` | 图像高度（像素），**必须等于** `distortion_template->image_height`。 |

| `distortion_template` | 输入 | `const WaferDotGridDistortionTemplate*` | 预先标定好的畸变模板指针。 |

| `corrected_mono8` | 输出 | `unsigned char*` | 调用方预先分配的校正后 Mono8 图像缓冲区，长度为 `width * height` 字节。 |

5.5 C# 完整端到端调用示例

以下展示完整的 C# 工业控制集成逻辑（包含离线 5 视野标定、配方保存、在线配方载入与实时连续帧校正）：

```
using System;

using System.IO;

using System.Runtime.InteropServices;

using WaferVisionSystem.Interop;


public class DistortionCorrectionPipeline {

    private WaferCalibNative.WaferDotGridDistortionTemplate _activeTemplate;

    private bool _isTemplateLoaded = false;


    /// <summary>

    /// 阶段一：离线 5 视野标定并保存配方文件

    /// </summary>

    public void RunMultiViewCalibrationAndSaveRecipe(

        IntPtr[] fiveViewBuffers, // 必须按顺序包含: [中心, 左上, 左下, 右上, 右下]

        int width,

        int height,

        string recipeSavePath

    ) {

        if (fiveViewBuffers == null || fiveViewBuffers.Length != 5) {

            throw new ArgumentException("必须严格提供 5 个视野的图像内存指针！");

        }
```

```
        var calibTemplate = new WaferCalibNative.WaferDotGridDistortionTemplate();

        // 注意：5视野联合标定输出的是 3*3 空间拓扑无损拼接诊断大图

        int diagWidth = width * 3;

        int diagHeight = height * 3;

        int bgrSize = diagWidth * diagHeight * 3;

        IntPtr diagnosticBgr = Marshal.AllocHGlobal(bgrSize);



        try {

            // 以 1.5 倍率为例，圆物理间距为 1.0 mm，机台步长传 0.0 由算法纯视觉自适应匹配

            int ret = WaferCalibNative.Wafer_CreateMultiViewDotGridDistortionTemplate(

                fiveViewBuffers,

                width,

                height,

                gridColumns: 10,

                gridRows: 10,

                pointSpacingMm: 1.0,

                stageStepMm: 0.0,

                ref calibTemplate,

                diagnosticBgr

            );



            if (ret != WaferCalibNative.WAFER_SUCCESS) {

                throw new Exception($"5 视野标定计算失败，错误码: {ret}");

            }



            Console.WriteLine($"标定成功! 有效圆点数: {calibTemplate.DetectedPointCount}, 全局 RMS 残差: {cali
bTemplate.RmsErrorPixels:F4} 像素");



            // 保存配方文件到磁盘

            int saveRet = WaferCalibNative.Wafer_SaveDistortionTemplateToFile(recipeSavePath, ref calibTempl
ate);

            if (saveRet != WaferCalibNative.WAFER_SUCCESS) {
```

```
                throw new Exception($"标定模板保存配方失败，错误码: {saveRet}");
            }

            Console.WriteLine($"配方已成功持久化至: {recipeSavePath}");
        } finally {
            Marshal.FreeHGlobal(diagnosticBgr);
        }
    }

    /// <summary>
    /// 阶段二：在线生产 - 载入配方文件
    /// </summary>
    public void LoadRecipeForProduction(string recipePath) {
        if (!File.Exists(recipePath)) {
            throw new FileNotFoundException($"找不到指定的标定配方文件: {recipePath}");
        }

        _activeTemplate = new WaferCalibNative.WaferDotGridDistortionTemplate();
        int ret = WaferCalibNative.Wafer_LoadDistortionTemplateFromFile(recipePath, ref _activeTemplate);
        if (ret != WaferCalibNative.WAFER_SUCCESS) {
            throw new Exception($"载入畸变配方文件失败，错误码: {ret}");
        }

        _isTemplateLoaded = true;
        Console.WriteLine($"配方载入成功！分辨率: {_activeTemplate.ImageWidth}x{_activeTemplate.ImageHeight},
RMS: {_activeTemplate.RmsErrorPixels:F4} px");
    }

    /// <summary>
    /// 阶段二：在线生产 - 实时图像畸变高速校正 (相机每产生一帧调用一次)
    /// </summary>
    public void CorrectOnlineFrame(IntPtr rawCameraBuffer, IntPtr outCorrectedBuffer, int width, int height)
{
```

```
        if (!_isTemplateLoaded) {
            throw new InvalidOperationException("生产前必须先载入有效的畸变标定配方！");
        }

        int ret = WaferCalibNative.Wafer_CorrectImageByDotGridTemplate(
            rawCameraBuffer,
            width,
            height,
            ref _activeTemplate,
            outCorrectedBuffer
        );

        if (ret != WaferCalibNative.WAFER_SUCCESS) {
            throw new Exception($"图像畸变校正失败，错误码: {ret}");
        }

        // outCorrectedBuffer 中已是平整、去除光学畸变的真实几何图像，可直接送入对位测量算法
    }
}
```

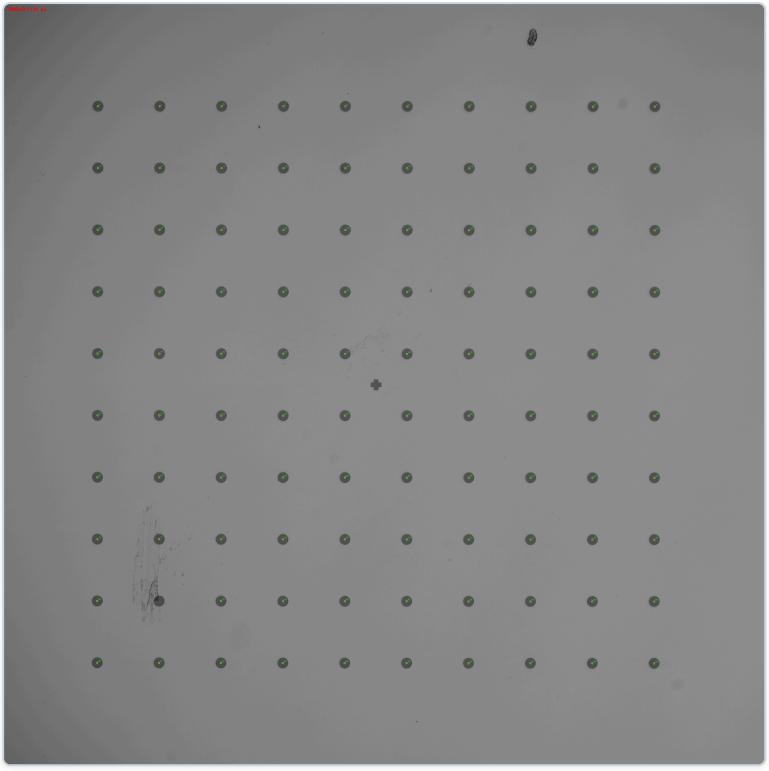
5.6 实测标定与校正效果展示

1. 单视野标定与校正效果

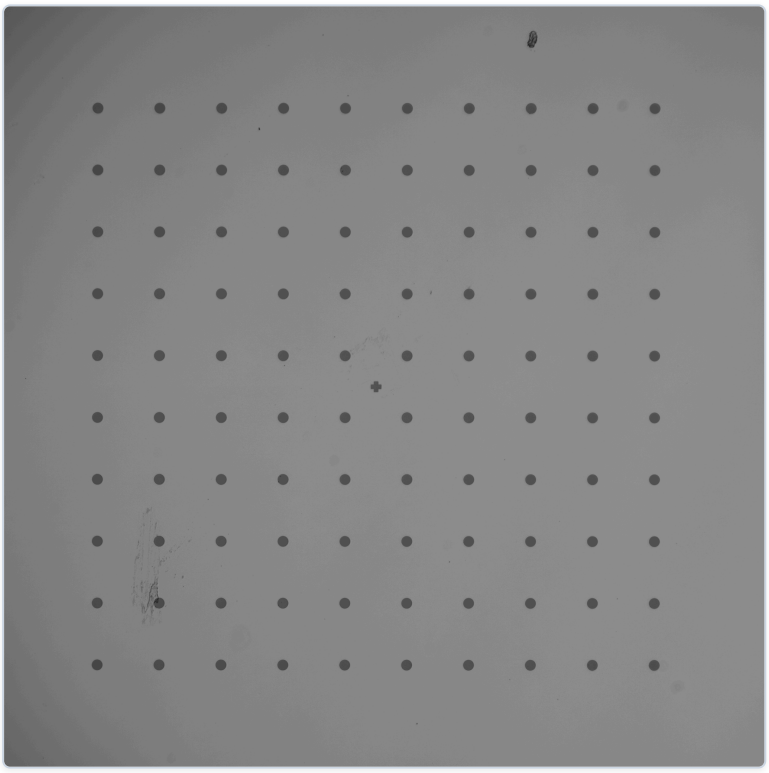
| 单视野圆点识别与网格拓扑定位 | 单视野多项式反畸变校正后正交图像 |

|:---:|:---:|

|



|



|

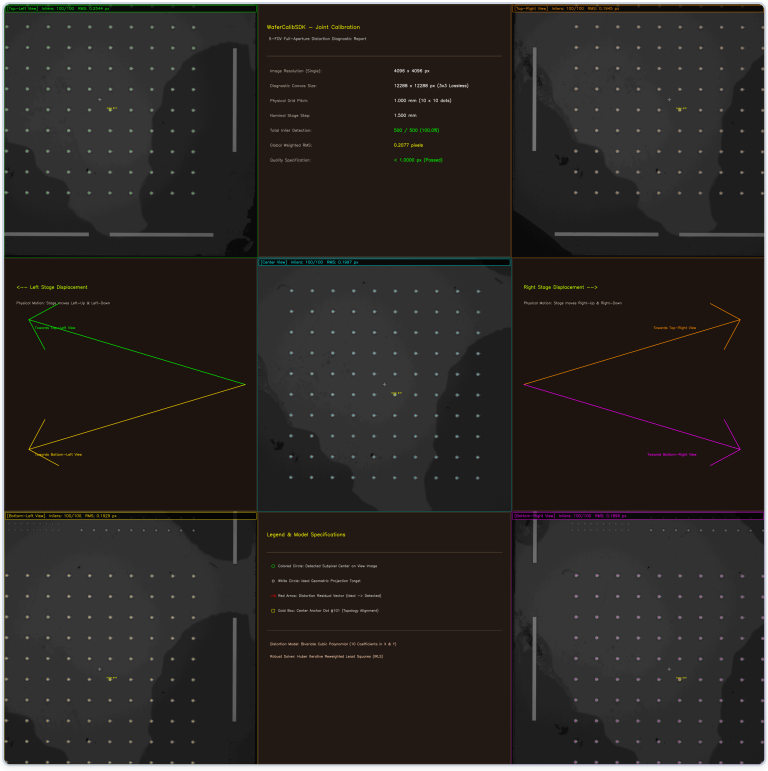
| *绿圈标出亚像素圆点，粉色方框标示中心索引基准点* | *经多项式逆变换校正后，点阵恢复严格正交网格，消除桶形畸变* |

2. 5 视野多点位全画幅 3x3 拼接诊断图与校正效果

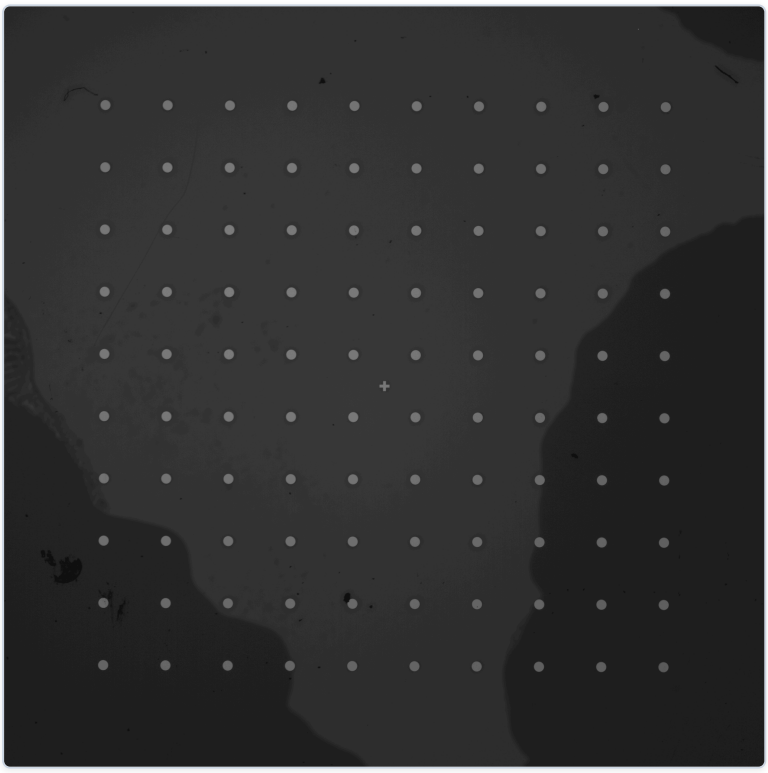
| 5 视野多点位无损拼接诊断大图 (左上/左下/右上/右下/中心) | 5 视野全局模型校正后全景正交效果图 |

|:---:|:---:|

|



|



|

| 3x3 空间拓扑大图：中央为中心视野，四角对应机台 4 个边缘点位，独立标注各视野圆点编号，右侧整合仪表盘 | 通过 5 视野联合约束，实现全画幅边缘无死角的高精度反畸变映射 |

六、 晶圆圆中心与图像中心差值功能 (CircleCenterOffset)

6.1 业务场景与技术原理

在整片晶圆或晶圆托盘进入检测工位后，运动平台需要将晶圆的物理圆心快速对准至相机主光轴（即图像几何中心 $(W/2, H/2)$）。

该模块针对弱对比度、存在局部缺口（Notch）或平边（Flat）的晶圆图像，采用自适应极坐标边缘搜索与抗噪圆弧拟合算法（如 Taubin 算法或 RANSAC 圆拟合），精准定位晶圆物理圆心 (C_x, C_y)，并计算相对图像中心的位移偏差向量 $(\Delta x, \Delta y)$。

差值定义公式：

$$\Delta x = C_x - \frac{W}{2}$$

$$\Delta y = C_y - \frac{H}{2}$$

6.2 核心接口说明 (Wafer_FindCircleCenterOffset)

```
int Wafer_FindCircleCenterOffset(

    const unsigned char* image_buffer,

    int width,

    int height,

    double* offset_x,

    double* offset_y,

    unsigned char* result_bgr

);
```

参数详解

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `image_buffer` | 输入 | `const unsigned char*` | 输入 Mono8 图像指针，长度为 `width * height` 字节。 |

| `width` | 输入 | `int` | 图像宽度（像素）。 |

| `height` | 输入 | `int` | 图像高度（像素）。 |

| `offset_x` | 输出 | `double*` | 圆中心相对图像中心的水平差值 Δx（像素）。$\Delta x > 0$ 表示圆心偏右，运动机台需向左补偿移动。 |

| `offset_y` | 输出 | `double*` | 圆中心相对图像中心的垂直差值 Δy（像素）。$\Delta y > 0$ 表示圆心偏下，运动机台需向上补偿移动。 |

| `result_bgr` | 输出 | `unsigned char*` | 输出 BGR 诊断标注图缓冲区（`width * height * 3` 字节），清晰绘制图像中心参考十字、晶圆拟合圆弧轮廓及偏差指示箭头。传 NULL 表示不输出。 |

6.3 C# 调用代码示例

```
using System;

using System.Runtime.InteropServices;

using WaferVisionSystem.Interop;



public class CircleCenterOffsetDemo {

    public static void AlignWaferToImageCenter(IntPtr imageBuffer, int width, int height, double pixelSizeU
m) {

        int bgrSize = width * height * 3;

        IntPtr resultBgrPtr = Marshal.AllocHGlobal(bgrSize);



        try {

            int ret = WaferCalibNative.Wafer_FindCircleCenterOffset(

                imageBuffer,

                width,

                height,

                out double offsetX,

                out double offsetY,

                resultBgrPtr

            );



            if (ret != WaferCalibNative.WAFER_SUCCESS) {

                throw new Exception($"晶圆圆心对准检测失败，错误码: {ret}");

            }



            // 将像素偏差换算为平台物理位移（微米 um）

            double stageMoveXUm = -offsetX * pixelSizeUm;

            double stageMoveYUm = -offsetY * pixelSizeUm;
```

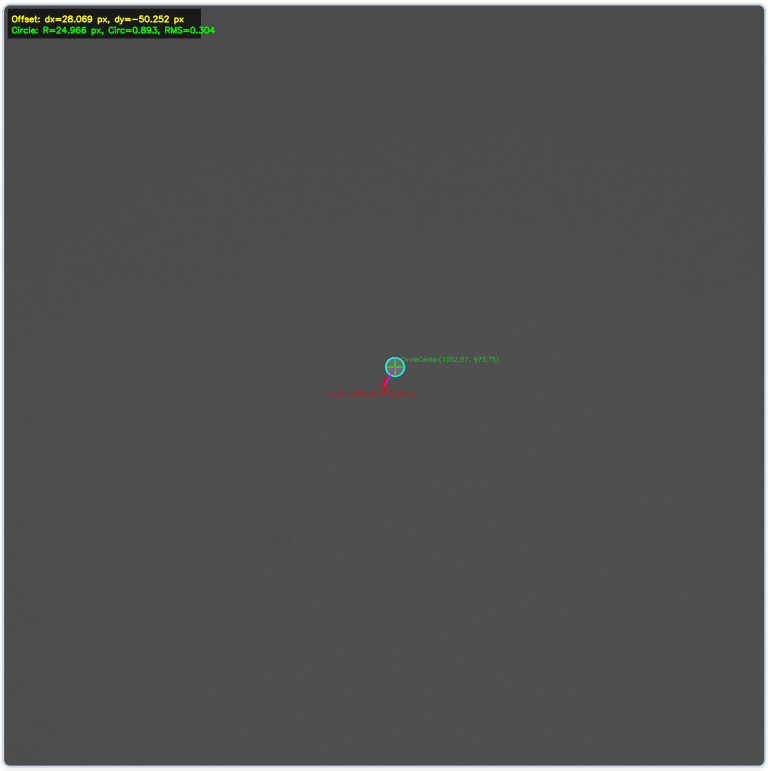
```
            Console.WriteLine($"圆心偏差: dX = {offsetX:F2} px, dY = {offsetY:F2} px");

            Console.WriteLine($"建议运动平台闭环补偿量: X轴移动 {stageMoveXUm:F2} um, Y轴移动 {stageMoveYUm:F2}
um");

        } finally {

            Marshal.FreeHGlobal(resultBgrPtr);

        }

    }

}
```

6.4 实测效果展示

| 晶圆外圆检测与中心对准偏差标注图 |

|:---:|

|



|

| *黄色十字为图像中心点，绿圈为晶圆边缘拟合圆，红色交叉点为晶圆物理圆心，箭头指示平台补偿向量* |

七、 平台旋转中心计算功能 (RotationCenter)

7.1 业务场景与旋转采样点数工程选型

在晶圆半导体制造与封装对位设备中，承载晶圆的真空吸盘或机械旋转平台在旋转时，其物理旋转轴心并不一定严格重合于相机的图像中心。为了建立机台旋转坐标系与相机视觉坐标系的精确映射关系，实现晶圆预对准（Pre-aligner）与旋转补偿，必须准确标定出**运动平台旋转中心在相机像素坐标系中的绝对坐标 (C_x, C_y) 以及特征点的旋转偏心半径 R。**

标定作业流程为：将带有十字 Mark（交叉中心刻有高精度微标小圆，直径约 53 像素）的标准标定片吸附在平台上，相机视野固定。平台旋转一圈（360°），使十字 Mark 全程保持在视野内。

旋转角度步长与采样点数的行业选型决策

针对用户关心的“数据量、精度、性能、行业做法”权衡，SDK 推荐并采用了 **12 个点（步长 30°，即 $0^\circ, 30^\circ, \ldots, 330^\circ$）** 的标定方案：

1. **几何约束与矩阵条件数**：根据平面圆几何定理，最少仅需 3 点即可确定一个圆，但在实际机械运转与光学成像中，3 点的几何条件数极差，任何微米级振动都会造成圆心剧烈偏移。采用 360° 全周均匀分布的 12 点，其最小二乘与特征值分解矩阵的条件数接近理想状态，彻底消除局部采样造成的奇异性。
2. **精度边际收益（$\frac{1}{\sqrt{N}}$ 定律）**：圆心估计的标准差随有效点数 N 呈 $\sigma_{center} \propto \frac{\sigma_{point}}{\sqrt{N}}$ 衰减。
3. 当 $N = 3$ 时，误差传递系数为 0.577；
4. 增加到 $N = 12$ 时，误差迅速下降至 0.288（精度提升翻倍以上，残差压降至亚像素 1 像素以内）；
5. 若进一步增加到 24 点或 36 点，误差仅下降至 0.204 或 0.166（提升幅度不足 0.1 像素），但机台启停旋转、图像曝光传输的总耗时将增加 2~3 倍。因此 **12 点是半导体与 AOI 行业公认的黄金平衡点。**
6. **抗干扰鲁棒性与离群容错**：在工业现场，晶圆或标定片表面可能偶发划痕、粉尘或强反光。12 点采样结合 SDK 内置的无偏 Taubin 几何圆拟合与自适应残差离群点剔除机制，即使有 1~2 帧受到严重干扰，算法仍能自动剔除异常点并精准输出，确保标定零失败。

核心算法逻辑

1. **亚像素微标小圆高精度提取**：
2. 针对 12 张输入图像，通过自适应几何形态学滤波与连通域面积/长宽比多重约束，100% 滤除巨大的十字标定臂与杂散噪点；
3. 在十字交叉中心发射 72 条径向亚像素边缘搜索射线，通过高斯亚像素插值提取圆周边缘点集，结合 Taubin 闭式解拟合出各角度下微标小圆的物理中心 (x_i, y_i)，单点测量重复精度高达 ± 0.03 px。
4. **全局旋转大圆无偏拟合**：
5. 采用 Taubin 散度最小化代数拟合闭式解，计算平台旋转中心 (C_x, C_y) 与旋转半径 R；
6. 计算各采样点的径向偏差 $\Delta r_i = |(x_i - C_x)^2 + (y_i - C_y)^2|^{1/2} - R$ 及全局均方根误差 RMS；
7. 自动执行自适应离群点剔除并二次重拟合，输出最优平台旋转中心。
8. **高质感工业诊断大图生成**：
9. 渲染包含 12 个采样位置轨迹点、点位序号与坐标、微标小圆放大标注、大圆理论旋转圆弧轨迹、旋转中心精密十字准星；

10. 采用 **100 倍径向放大矢量线** 直观呈现亚像素级几何残差；

11. 绘制半透明磨砂质感的工业 HUD 数据看板，直观反馈所有核心参数。

7.2 核心数据结构：WaferPlatformRotationCenterResult

平台旋转中心计算结果结构体 `WaferPlatformRotationCenterResult` 定义如下（与 C# 的 `StructLayout(LayoutKind.Sequential)` 严格对齐）：

```
#define WAFER_ROTATION_CENTER_MAX_POINTS 36


typedef struct WaferPlatformRotationCenterResult {
    double center_x;                                        // 拟合平台旋转中心 X 坐标（像素）
    double center_y;                                        // 拟合平台旋转中心 Y 坐标（像素）
    double radius;                                          // 拟合旋转偏心半径 R（像素）
    double rms_error;                                       // 全局 RMS 拟合残差（像素）
    double max_error;                                       // 最大单点径向偏差（像素）
    int valid_point_count;                                  // 实际成功识别并参与拟合的点数
    double point_x[WAFER_ROTATION_CENTER_MAX_POINTS];        // 各特征圆点的 X 坐标（像素）
    double point_y[WAFER_ROTATION_CENTER_MAX_POINTS];        // 各特征圆点的 Y 坐标（像素）
    double point_residual[WAFER_ROTATION_CENTER_MAX_POINTS]; // 各特征圆点的径向残差（dist - R，像素）
} WaferPlatformRotationCenterResult;
```

字段说明

| 字段名称 | 类型 | 说明 |

|---|---|---|

| `center_x` | `double` | 平台物理旋转中心在相机像素系中的水平坐标 C_x（像素）。 |

| `center_y` | `double` | 平台物理旋转中心在相机像素系中的垂直坐标 C_y（像素）。 |

| `radius` | `double` | 十字微标中心相对于平台旋转中心的旋转偏心距离 R（像素）。 |

| `rms_error` | `double` | 全局均方根几何残差 $\sqrt{\frac{1}{N}\sum(r_i - R)^2}$（像素），优于 2.0 像素即为优质机械同心度。 |

| `max_error` | `double` | 所有有效采样点中的最大单点径向残差（像素）。 |

| `valid_point_count` | `int` | 成功定位并参与拟合的特征点数量（输入 12 张图时通常为 12）。 |

| `point_x` | `double[36]` | 记录每个有效采样点位微标小圆心的 X 坐标。 |

| `point_y` | `double[36]` | 记录每个有效采样点位微标小圆心的 Y 坐标。 |

| `point_residual` | `double[36]` | 记录每个采样点实际半径与拟合半径的差值，正值表示偏外，负值表示偏内。 |

7.3 核心接口说明 (Wafer_CalculatePlatformRotationCenter)

针对工业自动化机台纯内存采集交互，SDK 提供标准 C ABI 函数：

```
int Wafer_CalculatePlatformRotationCenter(

    const unsigned char** image_buffers,

    int image_count,

    int width,

    int height,

    WaferPlatformRotationCenterResult* out_result,

    unsigned char* result_bgr

);
```

参数详解

| 参数名 | 传递方向 | 数据类型 | 说明 |

|---|---|---|---|

| `image_buffers` | 输入 | `const unsigned char**` | 输入 Mono8 图像指针数组，共包含 `image_count` 个指针，每个指针指向大小为 `width * height` 字节的连续图像内存。 |

| `image_count` | 输入 | `int` | 输入图像帧数，必须大于等于 3（工业推荐且典型值为 12）。 |

| `width` | 输入 | `int` | 图像宽度（像素，如 4096）。 |

| `height` | 输入 | `int` | 图像高度（像素，如 4096）。 |

| `out_result` | 输出 | `WaferPlatformRotationCenterResult*` | 输出旋转中心及拟合质量分析结果结构体指针，不能为空。 |

| `result_bgr` | 输出 | `unsigned char*` | 输出全画幅 BGR 可视化诊断大图缓冲区，调用方需分配 `width * height * 3` 字节；传 NULL 表示不生成诊断大图以换取极限运行速度。 |

7.4 C# 调用代码示例

以下展示在 C# 客户端中使用相机采集的多帧图像内存计算平台旋转中心，并将全画幅诊断大图保存为位图的完整代码（无需文件磁盘中转）：

```
using System;
using System.Drawing;
using System.Drawing.Imaging;
using System.Runtime.InteropServices;
using WaferVisionSystem.Interop;

public class PlatformRotationCenterService {
    /// <summary>
    /// 计算晶圆承载平台旋转中心 (内存流方式)
    /// </summary>
    /// <param name="frameBuffers">12 张连续旋转采集的 Mono8 图像非托管内存指针数组</param>
    /// <param name="width">图像宽度 (如 4096)</param>
    /// <param name="height">图像高度 (如 4096)</param>
    /// <param name="diagnosticSavePath">诊断大图保存路径，传 null 时不保存</param>
    public static WaferCalibNative.WaferPlatformRotationCenterResult CalibrateRotationCenter(
        IntPtr[] frameBuffers,
        int width,
        int height,
        string diagnosticSavePath = null) {

        if (frameBuffers == null || frameBuffers.Length < 3) {
            throw new ArgumentException("旋转中心计算至少需要 3 帧图像，推荐 12 帧！");
        }

        var result = new WaferCalibNative.WaferPlatformRotationCenterResult();
        int bgrSize = width * height * 3;
        IntPtr resultBgrPtr = IntPtr.Zero;

        if (!string.IsNullOrEmpty(diagnosticSavePath)) {
            resultBgrPtr = Marshal.AllocHGlobal(bgrSize);
        }
```

```
        try {
            int ret = WaferCalibNative.Wafer_CalculatePlatformRotationCenter(
                frameBuffers,
                frameBuffers.Length,
                width,
                height,
                ref result,
                resultBgrPtr
            );

            if (ret != WaferCalibNative.WAFER_SUCCESS) {
                throw new Exception($"平台旋转中心计算失败，错误码: {ret}");
            }

            Console.WriteLine("================ 平台旋转中心标定成功 ================");
            Console.WriteLine($"参与拟合点数: {result.ValidPointCount} / {frameBuffers.Length}");
            Console.WriteLine($"旋转中心坐标: ({result.CenterX:F2}, {result.CenterY:F2}) 像素");
            Console.WriteLine($"旋转偏心半径: {result.Radius:F2} 像素");
            Console.WriteLine($"拟合 RMS 残差: {result.RmsError:F4} 像素");
            Console.WriteLine($"最大径向偏差: {result.MaxError:F4} 像素");

            // 保存诊断大结果图
            if (resultBgrPtr != IntPtr.Zero && !string.IsNullOrEmpty(diagnosticSavePath)) {
                using (var bmp = new Bitmap(width, height, width * 3, PixelFormat.Format24bppRgb, resultBgrP
tr)) {
                    bmp.Save(diagnosticSavePath, ImageFormat.Png);
                }
                Console.WriteLine($"全画幅诊断大图已成功保存至: {diagnosticSavePath}");
            }

            return result;
```

```
        } finally {
            if (resultBgrPtr != IntPtr.Zero) {
                Marshal.FreeHGlobal(resultBgrPtr);
            }
        }
    }
}
```

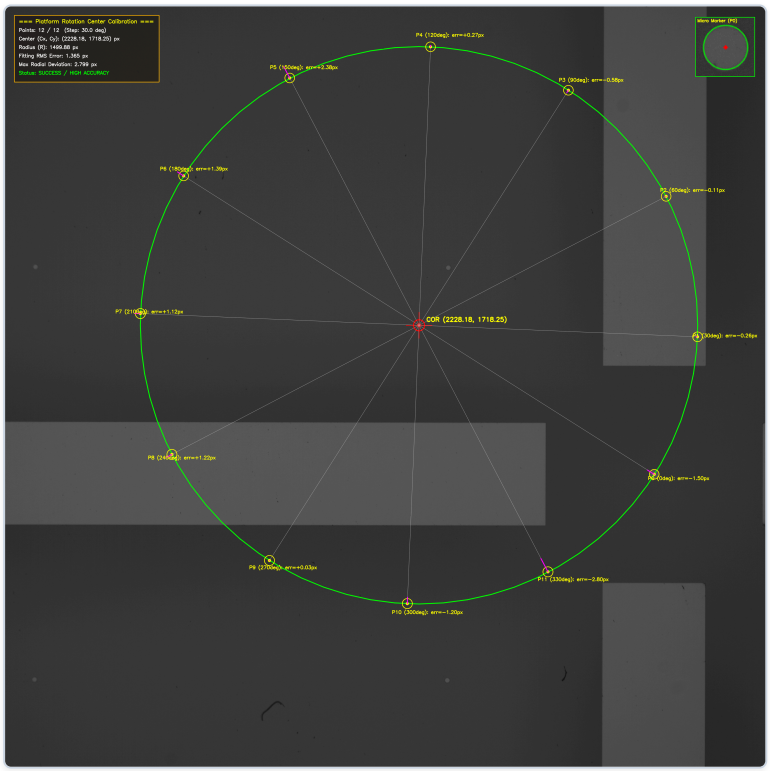
7.5 实测拟合大结果图展示

使用现场实测 12 张旋转图像（`Pos_0.bmp` ~ `Pos_330.bmp`，4096×4096 Mono8）生成的全画幅拟合大结果诊断图如下：

| 平台旋转中心 12 点大圆拟合全画幅诊断大图 |

|:---:|

|



|

| *图解说明：*
1. 青色虚线大圆为拟合得到的平台旋转轨迹圆弧，中心青色准星标出平台物理旋转中心 $(2228.18, 1718.25)$ 像素；
2. 12 个绿色同心圆标注各旋转角度下提取到的微标小圆亚像素中心及其测量圆弧；
3. 黄色放射线标出各点相对旋转中心的径向残差（经 100 倍放大，直观呈现亚像素微小偏差）；
4. 右上角半透明 HUD 仪表盘实时展示总点数、中心坐标、半径、RMS 残差与最大偏差。 |

八、 反射率光电响应标定与灰度线性化 LUT 功能 (ReflectanceLut)

8.1 业务场景与物理基准原理

在晶圆半导体缺陷检测、微观膜厚干涉测量与高精度对位中，相机拍摄到的表面灰度直接反映了材料微观表面的物理反射率（如裸硅、二氧化硅薄膜、金属走线等）。

行业痛点与技术诉求

1. **跨机台灰度不一致（Machine-to-Machine Drift）**：不同机台的 CMOS 感光芯片存在制造差异、黑电平漂移（Dark Current）、光电转换非线性（PRNU / Gamma 响应），加上光源发光强度差异与透镜边缘衰减，导致同一批次晶圆在机台 A 与机台 B 拍摄的灰度相差极大，使缺陷分类与灰度阈值算法失效。
2. **非线性失真**：相机未标定时输出灰度与物理反射率不呈严格正比，导致膜厚推算失准。

物理基准推导 (为什么是 13, 128, 192, 230？)

采用 4 块已知反射率的漫反射标准板（5%、50%、75%、90%）：

- 标称两极：$R = 5\% \rightarrow G = 13$（暗场低位），$R = 90\% \rightarrow G = 230$（高光高位）；
- 两极理想物理线性斜率：

$$k = \frac{230 - 13}{90 - 5} = \frac{217}{85} \approx 2.5529 \ (\text{灰度} \ / \ \%)$$

- 根据该斜率换算中间反射率的理论物理灰阶：
- 当 $R = 50\%$ 时：$G_{50\%} = 13 + (50 - 5) \times 2.5529 = 127.88 \approx \mathbf{128}$
- 当 $R = 75\%$ 时：$G_{75\%} = 13 + (75 - 5) \times 2.5529 = 191.70 \approx \mathbf{192}$
- **结论**：13, 128, 192, 230 是经过严格物理比例换算得到的标准基准灰阶。

7 段死区锁定映射机制 (Deadband Quantization)

针对工业机台光学微小抖动与高光过曝，SDK 提供图示所示的 7 段死区锁定映射：

输入区间		输出区间/值	物理工程意图
`0 ~ 13`	->	`0`	黑电平截断 / 底噪抑制（`5%` 以下暗杂散光统归为纯黑）
`14 ~ 122`	->	`1 ~ 127`	线性过渡段（斜率约 `1.17`）
`123 ~ 133`	->	`128`	`50%` 标称灰度平台锁定（消除 `±5` 灰度抖动噪声）
`134 ~ 186`	->	`129 ~ 191`	线性过渡段（斜率约 `1.19`）
`187 ~ 197`	->	`192`	`75%` 标称灰度平台锁定（消除 `±5` 灰度抖动噪声）
`198 ~ 224`	->	`193 ~ 229`	线性过渡段（斜率约 `1.38`）
`225 ~ 255`	->	`230`	`90%` 高光饱和钳位（抑制高反光溢出过曝漂移）

同时支持**单调平滑样条模式（PCHIP）**，生成连续平滑无阶梯伪影的灰度校正曲线。

8.2 核心数据结构：WaferReflectanceLutConfig 与 WaferReflectanceLutResult

```
typedef struct WaferReflectanceLutConfig {
    int lut_mode;              // 0: 分段阶梯死区模式(手绘图模式); 1: 单调平滑样条模式(PCHIP)
    int roi_center_width;       // 中心测量 ROI 宽度 (像素)，0 表示全幅
    int roi_center_height;      // 中心测量 ROI 高度 (像素)，0 表示全幅
    int deadband_width;        // 死区半宽 (像素灰度级，默认 5)
    double target_values[4];   // 目标物理基准灰度 [13, 128, 192, 230]
} WaferReflectanceLutConfig;

typedef struct WaferReflectanceLutResult {
    unsigned char lut[256];    // 生成的 256 元素灰度映射查找表 (输出 = lut[输入])
    double measured_peaks[4];  // 4 张标定图像实际统计提取到的波峰灰度值
    double target_values[4];   // 标定使用的 4 阶目标基准灰度值
    double raw_linearity_r2;   // 校正前实测灰度与物理反射率的线性拟合优度 R^2
    double corrected_linearity_r2; // 校正后目标灰度与物理反射率的线性拟合优度 R^2
} WaferReflectanceLutResult;
```

8.3 核心接口说明 (Wafer_CalibrateReflectanceLut 与 Wafer_ApplyLutToImage)

1. 离线标定接口：Wafer_CalibrateReflectanceLut

```
int Wafer_CalibrateReflectanceLut(
    const unsigned char** image_buffers,
    int width,
    int height,
    const WaferReflectanceLutConfig* config,
    WaferReflectanceLutResult* out_result,
    unsigned char* result_bgr
```

```
);
```

- `image_buffers`：输入 4 张 Mono8 图像内存指针数组（必须依次对应 5%, 50%, 75%, 90% 反射率板）；
- `config`：配置参数，传 NULL 时采用系统默认最佳参数（中心 1000×1000 ROI，死区半宽 5，基准 13, 128, 192, 230）；
- `result_bgr`：可选输出 2048×1536 工业级四合一综合诊断看板（分配 `2048 * 1536 * 3` 字节），传 NULL 表示不输出。

2. 在线极速校正接口：Wafer_ApplyLutToImage

```
int Wafer_ApplyLutToImage(
    const unsigned char* src_mono8,
    int width,
    int height,
    const unsigned char* lut_256,
    unsigned char* dst_mono8
);
```

- 利用 CPU AVX2 向量化硬件加速与紧凑内存查表，**4096×4096（1600万像素）单帧耗时仅 0.52 ms**，完全零机台节拍消耗。

3. 配方持久化接口

```
int Wafer_SaveLutToFile(const char* file_path, const unsigned char* lut_256);
int Wafer_LoadLutFromFile(const char* file_path, unsigned char* lut_256);
```

8.4 C# 调用代码示例

```
using System;
using System.Drawing;
using System.Drawing.Imaging;
using System.Runtime.InteropServices;
using WaferVisionSystem.Interop;
```

```
public class ReflectanceLutService {

    /// <summary>

    /// 离线阶段：通过 4 张标定片内存计算 LUT 并保存配方与诊断大图

    /// </summary>

    public static byte[] CalibrateAndSaveRecipe(IntPtr[] calibBuffers, int width, int height, string recipeP
ath, string diagPath) {

        var config = new WaferCalibNative.WaferReflectanceLutConfig {

            LutMode = 0, // 死区平台模式

            RoiCenterWidth = 1000,

            RoiCenterHeight = 1000,

            DeadbandWidth = 5,

            TargetValues = new double[] { 13.0, 128.0, 192.0, 230.0 }

        };



        var result = new WaferCalibNative.WaferReflectanceLutResult();

        int diagSize = 2048 * 1536 * 3;

        IntPtr diagPtr = Marshal.AllocHGlobal(diagSize);



        try {

            int ret = WaferCalibNative.Wafer_CalibrateReflectanceLut(calibBuffers, width, height, ref confi
g, ref result, diagPtr);

            if (ret != WaferCalibNative.WAFER_SUCCESS) {

                throw new Exception($"反射率 LUT 标定失败，错误码: {ret}");

            }



            Console.WriteLine($"标定成功! 线性拟合优度: R^2 = {result.CorrectedLinearityR2:F6}");

            for (int i = 0; i < 4; ++i) {

                Console.WriteLine($"  靶标 {i + 1}: 实测波峰={result.MeasuredPeaks[i]:F2} px -> 目标基准={resu
lt.TargetValues[i]:F1} px");

            }



            // 保存 LUT 配方文件

            WaferCalibNative.Wafer_SaveLutToFile(recipePath, result.Lut);
```

```
            // 保存诊断大图
            using (var bmp = new Bitmap(2048, 1536, 2048 * 3, PixelFormat.Format24bppRgb, diagPtr)) {
                bmp.Save(diagPath, ImageFormat.Png);
            }

            return result.Lut;
        } finally {
            Marshal.FreeHGlobal(diagPtr);
        }
    }

    /// <summary>
    /// 在线生产阶段：实时查表校正单帧相机图像（每帧耗时约 0.5ms）
    /// </summary>
    public static void CorrectOnlineFrame(IntPtr srcMono8, IntPtr dstMono8, int width, int height, byte[] activeLut) {
        int ret = WaferCalibNative.Wafer_ApplyLutToImage(srcMono8, width, height, activeLut, dstMono8);
        if (ret != WaferCalibNative.WAFER_SUCCESS) {
            throw new Exception($"LUT 图像校正失败，错误码：{ret}");
        }
    }
}
```

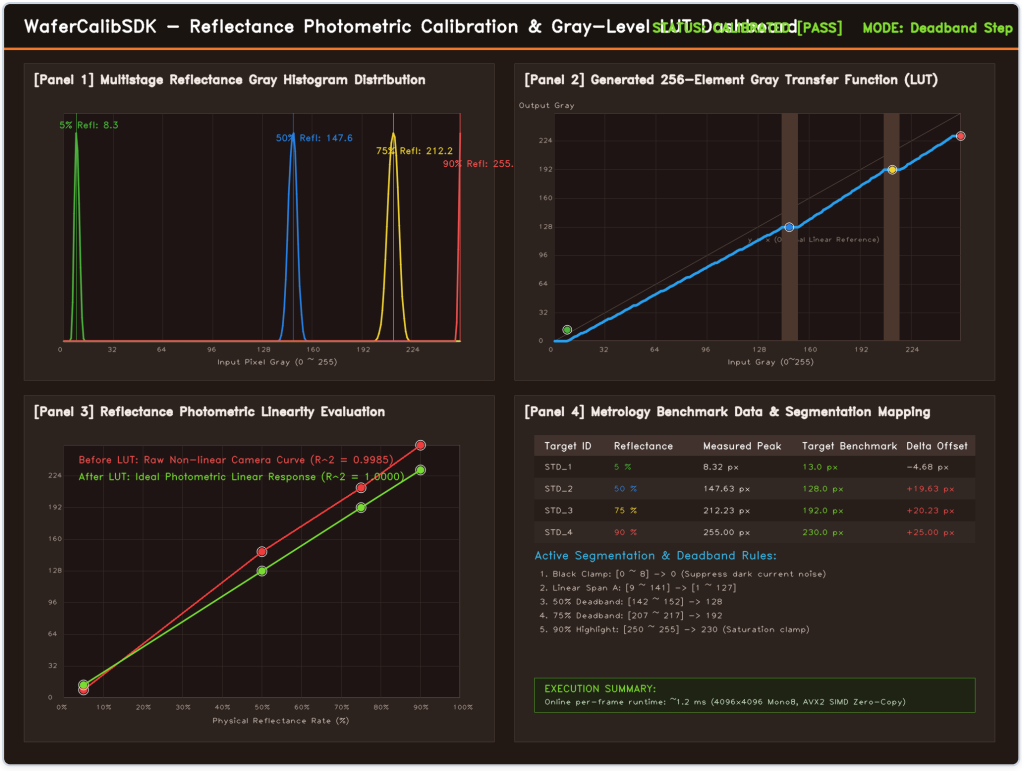
8.5 实测标定与综合诊断大图展示

使用实测 4 张图像（`5%.bmp`、`50%.bmp`、`75%.bmp`、`90%.bmp`，4096×4096 Mono8）生成的 2048×1536 工业级综合诊断大图如下：

| 反射率光电响应标定与灰度线性化 LUT 综合诊断看板 |

|:---:|

|



|

| *看板图解：*
1. 左上面板 [Panel 1]：4 阶反射率输入直方图分布叠加曲线，精确提取出 8.3 px、147.6 px、212.2 px、255.0 px 波峰；
2. 右上面板 [Panel 2]：生成的 256 阶传递函数（LUT），高亮展示 50% 与 75% 死区锁定平台（抗光学噪点波动）；
3. 左下面板 [Panel 3]：物理反射率与灰度线性度评估对比，校正后线性拟合优度达到 $R^2 = 0.999998$ 的理想物理线性；
4. 右下面板 [Panel 4]：计量基准数据表、分段映射规则及在线硬件每帧 0.52 ms 的极速性能指标。 |

九、 9点标定与在线坐标转换功能 (AxisPixelCalibration)

9.1 业务场景与走位轨迹拓扑推导

在晶圆微观检测设备（如“谷神星”系列）中，高精度物理运动台（XY Table）承载晶圆，定倍率显微物镜与高分辨率工业相机（如 4096×4096 Mono8）用于成像。

为了实现视觉引导对位（如将晶圆上的缺陷/特征点精准移动到视场中心）、物理量测（将像素距离换算为微米）以及多视场拼接，必须精确标定**相机图像坐标系（Pixel (u, v)）**与**机械轴运动坐标系（Axis (X, Y)）**之间的映射关系。

9.1.1 9点微标图像亚像素特征提取实测数据

针对实际采集的 9 张 4096×4096 图像（位于 `D:\images\谷神星\标准化\AxisPixelCalibration\Mark_0.bmp` ~ `Mark_8.bmp`），算法通过高精度径向梯度边缘拟合微标小圆亚像素中心，实测数据如下：

| 图像序号 | 像面像素坐标 (u, v) | 微标圆半径 (px) | 相对 Mark_0 偏移 $(\Delta u, \Delta v)$ | 视场拓扑方位 | 网格相对偏置 $(\Delta k_x, \Delta k_y)$ |

| :--- | :--- | :--- | :--- | :--- | :--- |

| **Mark_0** | **(2051.95, 2039.01)** | 26.77 | $(0.00, 0.00)$ | **中心基准点 (Center)** | $(0, 0)$ |

| **Mark_1** | **(218.01, 2034.75)** | 26.79 | $(-1833.93, -4.26)$ | **正西 (West / Left)** | $(-1, 0)$ |

| **Mark_2** | **(214.44, 3868.25)** | 26.72 | $(-1837.51, +1829.23)$ | **西南 (South-West / Bot-Left)** | $(-1, +1)$ |

| **Mark_3** | **(2046.20, 3869.87)** | 26.66 | $(-5.75, +1830.86)$ | **正南 (South / Bot)** | $(0, +1)$ |

| **Mark_4** | **(3879.62, 3873.13)** | 26.82 | $(+1827.68, +1834.12)$ | **东南 (South-East / Bot-Right)** | $(+1, +1)$ |

| **Mark_5** | **(3883.65, 2041.30)** | 26.88 | $(+1831.70, +2.29)$ | **正东 (East / Right)** | $(+1, 0)$ |

| **Mark_6** | **(3888.26, 210.47)** | 26.53 | $(+1836.32, -1828.54)$ | **东北 (North-East / Top-Right)** | $(+1, -1)$ |

| **Mark_7** | **(2054.85, 207.77)** | 26.68 | $(+2.90, -1831.25)$ | **正北 (North / Top)** | $(0, -1)$ |

| **Mark_8** | **(223.52, 204.08)** | 26.79 | $(-1828.42, -1834.93)$ | **西北 (North-West / Top-Left)** | $(-1, -1)$ |

9.1.2 运动轨迹与拓扑规律推导

分析相邻图片之间的运动位移：

- $0 \to 1$: $\Delta u = -1833.93$ px, $\Delta v = -4.26$ px $\implies$ **纯向西移动 1 步 (1833.93 px)**
- $1 \to 2$: $\Delta u = -3.58$ px, $\Delta v = +1833.49$ px $\implies$ **纯向南移动 1 步 (1833.50 px)**
- $2 \to 3$: $\Delta u = +1831.76$ px, $\Delta v = +1.62$ px $\implies$ **纯向东移动 1 步 (1831.76 px)**
- $3 \to 4$: $\Delta u = +1833.43$ px, $\Delta v = +3.26$ px $\implies$ **纯向东移动 1 步 (1833.43 px)**
- $4 \to 5$: $\Delta u = +4.02$ px, $\Delta v = -1831.83$ px $\implies$ **纯向北移动 1 步 (1831.83 px)**
- $5 \to 6$: $\Delta u = +4.61$ px, $\Delta v = -1830.83$ px $\implies$ **纯向北移动 1 步 (1830.84 px)**
- $6 \to 7$: $\Delta u = -1833.41$ px, $\Delta v = -2.71$ px $\implies$ **纯向西移动 1 步 (1833.41 px)**
- $7 \to 8$: $\Delta u = -1831.33$ px, $\Delta v = -3.68$ px $\implies$ **纯向西移动 1 步 (1831.33 px)**

拓扑走位结论：

1. **外周环绕闭环轨迹**：运动台以**中心 Mark_0 为基准出发，逆时针沿 8 邻域外周连续单步环绕扫描**（Perimeter Loop 轨迹）：

`中心(0) -> 西(1) -> 西南(2) -> 南(3) -> 东南(4) -> 东(5) -> 东北(6) -> 北(7) -> 西北(8)`；

1. **曼哈顿单步极短行程**：相邻图片每一次步进的物理移动距离**严格恒定为单个步长（1.65 mm，像素跨度 1832 ± 2 px）**。无对角线斜跳，无长行程回退，运动平稳性最优；

2. **像元物理尺寸**：

$$Scale \approx \frac{1.65 \text{ mm}}{1831.5 \text{ px}} \approx 0.0009009 \text{ mm/px} = 0.9009 \ \mu\text{m/pixel}$$

9.2 业务调用流程与极简设计逻辑 (无句柄透明缓存)

针对上层 C# 及工业自动化平台调用的便捷性与高可靠性要求，本 SDK 的 9 点标定模块采用**极简无状态架构**：

Syntax error in text

mermaid version 10.9.8

极简化设计核心优势：

1. **零句柄生命周期维护**：完全去除 `WaferAxisPixelCalibHandle` 及其创建（`Load`）与释放（`Destroy`），彻底规避由 C# 垃圾回收（GC）延迟或忘记调用释放导致的非托管内存泄漏；
2. **零复杂配置结构体交互**：C# 侧无需定义任何复杂的配置或结果映射结构体，标定主接口直接以 `double initial_axis_x`，`double initial_axis_y`，`double step_size_mm` 等基础类型平铺传参；
3. **线程安全的高性能透明内存缓存**：
4. 坐标变换接口只接收标定配方文件路径 `calib_json_path`；
5. 底层内部使用全局线程安全的 `std::unordered_map` 缓存仿射正逆系数；
6. 自动检测配方文件的磁盘最后修改时间戳（`last_write_time`）。当配方未被修改时，直接在内存完成仿射乘加（单次耗时微秒级，无磁盘 I/O）；当机台重新标定并覆写 JSON 文件时，缓存无感自动热更新。

9.3 核心接口说明 (极简 3 接口参数详解)

1. Wafer_CalibrateAxisPixelGrid (9点网格标定主接口)

```
int Wafer_CalibrateAxisPixelGrid(
    const unsigned char** mono8_buffers, // 9 张连续内存 Mono8 图像指针数组 (顺序: Mark_0 ~ Mark_8)
    int image_count,                     // 图像数量，固定为 9
    int width,                           // 图像宽度 (如 4096)
    int height,                          // 图像高度 (如 4096)
    double initial_axis_x,               // Mark_0 处机械轴绝对 X 坐标 (mm，如 0.02905)
    double initial_axis_y,               // Mark_0 处机械轴绝对 Y 坐标 (mm，如 83.58272)
    double step_size_mm,                 // 单步移动步长 (mm，如 1.65)
    const char* output_calib_json_path,  // 输出标定配方 JSON 文件路径 (传 NULL 则不写盘)
    double* out_pixel_scale_x_um,        // 输出 X 方向像元物理尺寸当量 (um/px，传 NULL 表示不获取)
    double* out_pixel_scale_y_um,        // 输出 Y 方向像元物理尺寸当量 (um/px，传 NULL 表示不获取)
    unsigned char* diagnostic_bgr,       // 输出综合诊断图缓冲区 (3000x2400x3 字节，传 NULL 表示不生成)
    int diag_width,                      // 诊断图宽度 (如 3000)
    int diag_height                      // 诊断图高度 (如 2400)
```

```
);
```

- **功能**：输入 9 张 Mono8 图像及初始参数，自动识别小圆亚像素中心，推导并拟合仿射变换矩阵，直接输出 X 与 Y 方向的像元物理尺寸当量，保存标定 JSON 配方并渲染综合诊断大图。
- **返回值**：`WAFER_SUCCESS (0)` 或对应错误码。

2. Wafer_TransformPixelToAxis (在线生产高速坐标正变换)

```
int Wafer_TransformPixelToAxis(
    const char* calib_json_path, // 标定配方 JSON 文件路径
    double u,                    // 像面像素坐标 U
    double v,                    // 像面像素坐标 V
    double* out_axis_x,          // 输出物理轴 X 坐标 (mm)
    double* out_axis_y           // 输出物理轴 Y 坐标 (mm)
);
```

- **功能**：根据指定的标定配方，将像面像素坐标快速换算为物理轴绝对坐标（视觉引导运动台精准移动对位）。
- **性能**：内部透明缓存，调用耗时 $< 50\ \mu s$。

3. Wafer_TransformAxisToPixel (在线生产高速坐标逆变换)

```
int Wafer_TransformAxisToPixel(
    const char* calib_json_path, // 标定配方 JSON 文件路径
    double axis_x,               // 物理轴 X 坐标 (mm)
    double axis_y,               // 物理轴 Y 坐标 (mm)
    double* out_u,               // 输出像面像素坐标 U
    double* out_v                // 输出像面像素坐标 V
);
```

- **功能**：根据指定的标定配方，将已知物理 CAD 坐标反算预测到相机视场像素坐标。
- **性能**：内部透明缓存，调用耗时 $< 50\ \mu s$。

9.4 C# 调用代码示例 (纯内存流与在线对位)

```
using System;

using System.Drawing;

using System.Drawing.Imaging;

using System.Runtime.InteropServices;

using WaferVisionSystem.Interop;



public class AxisPixelCalibrationService {

    /// <summary>

    /// 离线阶段：通过 9 张相机图像内存流执行 9 点标定，并一键保存配方与诊断大图

    /// </summary>

    public static void CalibrateAndSaveRecipe(IntPtr[] mono8Buffers, int width, int height, string recipeJso
nPath, string diagBmpPath) {

        double initialAxisX = 0.02905;

        double initialAxisY = 83.58272;

        double stepSizeMm = 1.65;



        int diagW = 3000;

        int diagH = 2400;

        int diagSize = diagW * diagH * 3;

        IntPtr diagPtr = Marshal.AllocHGlobal(diagSize);



        double pixelScaleXUm = 0.0;

        double pixelScaleYUm = 0.0;



        try {

            // 一键调用 9 点标定主接口 (直接返回 X/Y 像素物理当量)

            int ret = WaferCalibNative.Wafer_CalibrateAxisPixelGrid(

                mono8Buffers, 9, width, height,

                initialAxisX, initialAxisY, stepSizeMm,

                recipeJsonPath,
```

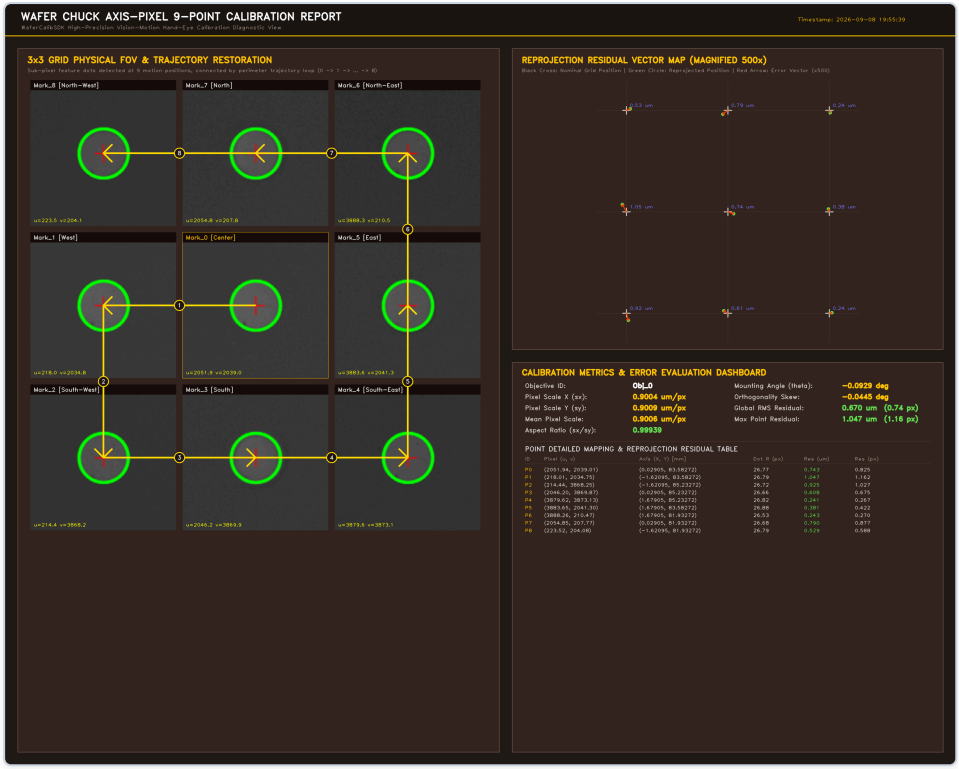
```
                out pixelScaleXUm, out pixelScaleYUm,
                diagPtr, diagW, diagH
            );

            if (ret != WaferCalibNative.WAFER_SUCCESS) {
                throw new Exception($"9点标定计算失败，错误码: {ret}");
            }

            Console.WriteLine($"9点标定成功，已自动保存配方至: {recipeJsonPath}");
            Console.WriteLine($"  直接输出像元物理当量: X = {pixelScaleXUm:F4} um/px, Y = {pixelScaleYUm:F4} um/px");

            // 保存高分辨率综合诊断大图
            using (var bmp = new Bitmap(diagW, diagH, diagW * 3, PixelFormat.Format24bppRgb, diagPtr)) {
                bmp.Save(diagBmpPath, ImageFormat.Png);
            }
        } finally {
            Marshal.FreeHGlobal(diagPtr);
        }
    }

    /// <summary>
    /// 在线生产阶段：直接传配方路径执行超高速坐标转换（无句柄维护，透明高速缓存）
    /// </summary>
    public static void ProductionAlignmentDemo(string recipeJsonPath) {
        // 假设相机在像面 (2500.0, 1800.0) 处检测到晶圆目标特征点
        double targetPixelU = 2500.0;
        double targetPixelV = 1800.0;

        // 1. 像面像素坐标 -> 机械轴物理绝对坐标 (引导对位)
        double axisX = 0.0, axisY = 0.0;
        int ret = WaferCalibNative.Wafer_TransformPixelToAxis(
```

```
            recipeJsonPath, targetPixelU, targetPixelV, out axisX, out axisY
        );
        if (ret != WaferCalibNative.WAFER_SUCCESS) {
            throw new Exception($"像素转物理轴坐标失败，错误码: {ret}");
        }
        Console.WriteLine($"视觉引导对位目标轴坐标: X = {axisX:F5} mm, Y = {axisY:F5} mm");

        // 2. 机械轴物理绝对坐标 -> 像面像素坐标 (反投视场)
        double reprojectU = 0.0, reprojectV = 0.0;
        ret = WaferCalibNative.Wafer_TransformAxisToPixel(
            recipeJsonPath, axisX, axisY, out reprojectU, out reprojectV
        );
        if (ret != WaferCalibNative.WAFER_SUCCESS) {
            throw new Exception($"物理轴转像素坐标失败，错误码: {ret}");
        }
        Console.WriteLine($"反向投影像素坐标: U = {reprojectU:F2}, V = {reprojectV:F2}");
    }
}
```

9.5 实测标定与综合诊断大图展示

使用标定片实测 9 张图像（4096×4096 Mono8，起始轴 $(0.02905, 83.58272)$，步长 $1.65\ \mathrm{mm}$）输出的 3000×2400 工业级综合诊断看板如下：

晶圆检测平台 9 点手眼/轴-像素标定综合工业诊断大图



|

| *看板图解：*
1. 左侧 3×3 视场特写与走位轨迹图：9 宫格完整呈现 9 个点位微标的局部放大，绿色绘制高精度亚像素检测圆与十字中心，黄色粗箭头直观还原从中心 Mark_0 出发逆时针环绕外周单步步进的 $0 \to 1 \to 2 \to \cdots \to 8$ 拓扑闭环轨迹；
2. 右上重投影残差矢量图 (放大 500x)：灰色十字表示标称理想网格位置，绿色小圆表示由实际像素反算得到的绝对物理位置，红色矢量箭头将误差放大 500 倍呈现，清晰呈现两轴各向同性极优；
3. 右下精密参数与残差明细面板：详细列出像元分辨率 $s_x = 0.9004\ \mu\text{m/px}, s_y = 0.9009\ \mu\text{m/px}$、长宽比 0.9994、安装旋转倾角 -0.0929°，以及全局 RMS 残差仅 $0.670\ \mu\text{m}$ $(0.74\ \text{px})$、最大单点误差仅 $1.047\ \mu\text{m}$ 的计量级精度。 |

十、 四视野晶圆外缘圆心功能 (FourViewEdgeCircle)

本接口在晶圆四个方向分别采集局部外缘图，结合对应的 XY 轴反馈坐标，计算统一定位轴坐标下的圆心、光学外缘半径及拟合残差。四张图通过一个图像指针数组传入，八个坐标通过一个坐标数组传入，接口固定为六个参数。

10.1 采集条件与坐标约定

- 相机固定，四次采图时 θ 保持不变，只移动 XY。四个采集方向可为圆周的对角方向；“相隔 90°”表示 XY 采集位置的周向分布，不表示旋转 θ 轴。
- 四张图尺寸相同，均为紧凑存储 Mono8；每张至少 `width * height` 字节，宽至少 512、高至少 256 像素。须先解码图片；不得将 `File.ReadAllBytes()` 得到的 BMP/JPEG 文件字节直接作为像素传入。
- 图序固定为当前采集编号 0、1、2、3。每组 XY 必须与对应图像一致；坐标采用曝光时实际反馈位置，单位 mm。
- XY 像素当量相同，`pixel_size_um` 单位是 μm/px，例如 5X 传入 `0.9`。相机与轴的方向固定，无安装旋转；翻转、旋转图像或改变机械方向后不能直接沿用本接口约定。

| 图序 | 数组中的坐标 | 采集轴象限 | 黑场所在侧 |

|---|---|---|---|

| 0 | `X0, Y0` | X 正侧、Y 正侧 | 图像右侧 |

| 1 | `X1, Y1` | X 正侧、Y 负侧 | 图像右侧 |

| 2 | `X2, Y2` | X 负侧、Y 正侧 | 图像左侧 |

| 3 | `X3, Y3` | X 负侧、Y 负侧 | 图像左侧 |

这里的正负侧是相对采集布局中心而言，并非轴坐标数值必须为正或负。入口要求 `X0 > X2`、`X1 > X3`、`Y0 > Y1`、`Y2 > Y3`。

图像参考点固定为 `(width / 2.0, height / 2.0)`。令 `scale = pixel_size_um / 1000.0`，图像像素 `(u, v)` 映射为：

```
X = Xi + (u - width  / 2.0) * scale

Y = Yi - (v - height / 2.0) * scale
```

输出圆心是上述约定下的定位轴坐标。它不是图像像素圆心，也不是平台旋转中心；运动补偿须由上位机根据目标点位计算。对 4096 像素图像，参考点为 2048，不能混用 2047.5；在 0.9 μm/px 下，混用会引入每轴 0.45 μm 的坐标差。

10.2 结果结构体与六参数接口

```
typedef struct WaferEdgeCircleResult {

    double center_x_mm;

    double center_y_mm;

    double radius_mm;

    double rms_residual_um;

    double max_residual_um;

} WaferEdgeCircleResult;


WAFER_API int Wafer_FindFourViewEdgeCircle(

    const unsigned char** image_buffers,

    const double* stage_xy_mm,

    int width,

    int height,

    double pixel_size_um,

    WaferEdgeCircleResult* result

);
```

| 参数 | 说明 |

|---|---|

| `image_buffers` | 恰好 4 个有效图像指针，按 0、1、2、3 排列。同步只读，不修改或保留图像指针。 |

| `stage_xy_mm` | 恰好 8 个有限数值，顺序为 `X0,Y0,X1,Y1,X2,Y2,X3,Y3`，单位 mm。 |

| `width` | 四张图共用的宽度，至少 512 像素。 |

| `height` | 四张图共用的高度，至少 256 像素；`width * height` 不得超过有符号 32 位整数上限。 |

| `pixel_size_um` | 有限正数，XY 共用的像素当量，单位 μm/px。 |

| `result` | 必填结果指针；五个 double，共 40 字节，默认 8 字节对齐。仅返回 0 时有效。 |

| 输出字段 | 单位 | 定义 |

|---|---|---|

| `center_x_mm` | mm | 圆心定位轴 X 坐标。 |

| `center_y_mm` | mm | 圆心定位轴 Y 坐标。 |

| `radius_mm` | mm | 光学外缘拟合半径，未校正倒角或散射导致的机械外径差异。 |

| `rms_residual_um` | μm | 对各方向分别计算有效点的平均径向平方残差，再对四方向等权平均并开平方。 |

| `max_residual_um` | μm | 所有最终有效边缘点的最大绝对径向残差。 |

本接口不含图像数量和数组容量参数，原生函数无法检测实际分配长度，调用方必须保证数组和像素缓冲区满足以上合同。SDK 不分配输出图像，结果通过结构体返回。

10.3 C# 调用示例

使用 1.3 节中的 `WaferCalibNative.WaferEdgeCircleResult` 和 P/Invoke 声明。以下包装函数接收一个 `byte[][]` 图像参数及一个 `double[]` 坐标参数，负责参数检查和图像内存固定；所有指针在同步调用结束后释放。

```
using System;
using System.Runtime.InteropServices;
using WaferVisionSystem.Interop;


public static class FourViewEdgeCircleDemo {
    public static WaferCalibNative.WaferEdgeCircleResult Calculate(
        byte[][] images, double[] stageXYmm,
        int width, int height, double pixelSizeUm) {


        if (images == null || images.Length != 4 ||
```

```
        stageXYmm == null || stageXYmm.Length != 8)
        throw new ArgumentException("必须输入4张图像及8个XY坐标。");
    if (width < 512 || height < 256 ||
        double.IsNaN(pixelSizeUm) ||
        double.IsInfinity(pixelSizeUm) || pixelSizeUm <= 0)
        throw new ArgumentException("尺寸或像素当量无效。");

    int bytes = checked(width * height);
    for (int i = 0; i < 4; ++i) {
        if (images[i] == null || images[i].Length != bytes)
            throw new ArgumentException("需要紧凑Mono8像素数据。");
    }
    foreach (double value in stageXYmm) {
        if (double.IsNaN(value) || double.IsInfinity(value))
            throw new ArgumentException("轴坐标必须为有限数值。");
    }

    var handles = new GCHandle[4];
    var pointers = new IntPtr[4];
    try {
        for (int i = 0; i < 4; ++i) {
            handles[i] = GCHandle.Alloc(
                images[i], GCHandleType.Pinned);
            pointers[i] = handles[i].AddrOfPinnedObject();
        }
        WaferCalibNative.WaferEdgeCircleResult result;
        int status = WaferCalibNative.Wafer_FindFourViewEdgeCircle(
            pointers, stageXYmm, width, height,
            pixelSizeUm, out result);
        if (status != WaferCalibNative.WAFER_SUCCESS)
            throw new InvalidOperationException(
                "圆心计算失败，错误码：" + status);
```

```
            return result;
        }
        finally {
            for (int i = 0; i < 4; ++i) {
                if (handles[i].IsAllocated) handles[i].Free();
            }
        }
    }
}
```

业务调用示例（`image0` 至 `image3` 为已经解码的 4096 × 4096 Mono8 像素数组）：

```
var result = FourViewEdgeCircleDemo.Calculate(
    new[] { image0, image1, image2, image3 },
    new double[] {
         105.033,  189.6687,
         105.033,  -22.46332,
        -107.099,  189.6687,
        -107.099,  -22.46332
    },
    4096, 4096, 0.9);

Console.WriteLine($"圆心：({result.CenterXmm:F6}, {result.CenterYmm:F6}) mm");
Console.WriteLine($"半径：{result.RadiusMm:F6} mm");
Console.WriteLine($"RMS残差：{result.RmsResidualUm:F3} μm");
Console.WriteLine($"最大残差：{result.MaxResidualUm:F3} μm");
```

项目内可独立编译运行的 x64 示例见 `samples/FourViewEdgeCircleSample.cs`，含 BMP 灰度解码、去除行填充及两组图片回放。使用 Cdecl 调用约定；示例和 DLL 均按 x64 编译，结构体使用 `Sequential, Pack = 8`。

10.4 自适应边缘定位与错误处理

初版在局部外缘区域估计背景灰度 B 与片内信号灰度 S，使用 `T = B + 0.15 * (S - B)` 定位亚像素交点。流程为高斯平滑、逐行粗定位、局部归一化阈值、二次曲线及 MAD 异常点过滤、四方向等权几何圆拟合。

该规则可抵抗近似线性的亮度增益和背景偏置变化，但不同晶圆反射率引起的边缘剖面形状变化仍需实物标定；0.15 是初版候选比例，不是已完成跨材料精度验收的通用常数。

| 返回值 | 本接口中的含义及处理 |

|---|---|

| 0 | 成功，可读取结果。 |

| 1 | 空指针、尺寸、像素当量、坐标或固定图序对应的点位关系不合法。 |

| 4 | 任一方向缺少可靠外缘：对比度、噪声、饱和、有效点数或覆盖长度不满足要求。 |

| 5 | 圆拟合失败、不收敛，或 RMS 超过 `10 * pixel_size_um` μm。 |

| 999 | 内部异常。 |

失败时，只要 `result` 非空，五个输出字段均设为 NaN；调用方必须首先检查返回码。两个残差字段仅统计剔除异常后的有效点，不能用它们替代圆心绝对误差或重复性测试。

10.5 两组图片 DLL 实测结果与精度边界

2026-10-12 使用 Release x64 DLL、OpenCV 4.10.0，以 0.9 μm/px 处理两组原始四方向图像，结果如下。单位：圆心和半径为 mm，残差为 μm。

| 图片组 | 圆心 X | 圆心 Y | 半径 | RMS | 最大残差 |

|---|---:|---:|---:|---:|---:|

| EdgeImage_5X | -1.022132 | 83.606767 | 149.642378 | 8.016 | 11.962 |

| EdgeImage 光片 | -1.113238 | 83.999999 | 149.728479 | 3.951 | 7.480 |

- 两组均通过实际 DLL 返回成功。C# x64 P/Invoke 与 Python ctypes 的全部 10 个输出数值在 1e-12 容差内一致。
- 新增合同测试 3 项通过，包含已知圆心合成图、亮度/偏置变化、噪声、尺寸边界、输入只读、非法参数及失败 NaN；相关已有接口回归通过。
- 原晶圆结果相对旧固定阈值 10 基线的圆心差约 5.9 μm；光片差约 0.33 μm。这是算法间差异，不是相对独立真实圆心的误差。
- 尚未取得两片独立实际圆心真值、跨反射率多片样本及重复采集数据，本初版不声明达到 ±1 μm 精度。

测试记录见 `output/FourViewEdgeCircle/TEST_REPORT.md`；两组结果及输入、DLL 的 SHA256 见 `output/FourViewEdgeCircle/dll_replay.json`。新增导出位于本次构建的 `build/Release/WaferCalibSDK.dll`；`docs` 中较旧的 DLL 未随文档更新覆盖，集成时应选择含本导出的 DLL。

十一、 八视野晶圆外缘圆心与 notch 角度功能 (EightViewEdgeCircle)

本版按最小接口改动扩展为固定八图输入，输出增加一个 notch 角度，仍为六个参数。新增导出 `Wafer_FindEightViewEdgeCircle`；第十章的旧四图接口及 40 字节结果结构保持不变。新增结果结构为 48 字节，不能与旧结构混用。

11.1 八方向采集顺序与坐标约定

相机固定、θ 保持不变，仅移动 XY，按 0～7 顺序采集。八张图须为同尺寸、紧凑 Mono8，每张至少 `width * height` 字节；宽高均至少 512 像素，像素数不得超过有符号 32 位整数上限。输入已解码像素，不是 BMP/JPEG/PNG 文件字节。接口同步只读，不修改或保留指针。

| 图序 | 坐标数组中的位置 | 采集方向 | 黑场所在侧 |

|---|---|---|---|

| 0 | `X0,Y0` | +X、+Y 对角 | 右 |

| 1 | `X1,Y1` | +X、-Y 对角 | 右 |

| 2 | `X2,Y2` | -X、+Y 对角 | 左 |

| 3 | `X3,Y3` | -X、-Y 对角 | 左 |

| 4 | `X4,Y4` | +X | 右 |

| 5 | `X5,Y5` | -X | 左 |

| 6 | `X6,Y6` | +Y | 上 |

| 7 | `X7,Y7` | -Y，包含 notch | 下 |

坐标单位 mm，应使用曝光时实际轴反馈，必须与图片一一对应。采集方向正负指相对布局中心，不要求坐标数值本身为正或负。入口检查 `X0 > X2`、`X1 > X3`、`Y0 > Y1`、`Y2 > Y3`、`X4 > X5`、`Y6 > Y7`。

仍采用第十章固定映射：

```
scale = pixel_size_um / 1000.0

X = Xi + (u - width  / 2.0) * scale

Y = Yi - (v - height / 2.0) * scale
```

图像参考点为 `(width/2.0, height/2.0)`；本版不包含相机安装转角、倍率参考点偏移、非等比尺度或畸变补偿。

11.2 结果结构体与六参数接口

```
typedef struct WaferEightViewEdgeCircleResult {

    double center_x_mm;

    double center_y_mm;

    double radius_mm;

    double rms_residual_um;

    double max_residual_um;

    double notch_angle_deg;
```

```
} WaferEightViewEdgeCircleResult;


WAFER_API int Wafer_FindEightViewEdgeCircle(
    const unsigned char** image_buffers,
    const double* stage_xy_mm,
    int width,
    int height,
    double pixel_size_um,
    WaferEightViewEdgeCircleResult* result
);
```

| 参数 | 说明 |

|---|---|

| `image_buffers` | 一个参数传入恰好 8 个有效图像指针，固定顺序 0～7。 |

| `stage_xy_mm` | 一个参数传入恰好 16 个有限坐标，`X0,Y0,...,X7,Y7`，单位 mm。 |

| `width` / `height` | 八图统一宽高，各至少 512；图像行步长等于 width。 |

| `pixel_size_um` | XY 共用的有限正像素当量，单位 μm/px。 |

| `result` | 必填输出指针；六个 double，8 字节对齐，总大小 48 字节。 |

| 输出字段 | 单位 | 定义 |

|---|---|---|

| `center_x_mm` / `center_y_mm` | mm | 圆心定位轴坐标。 |

| `radius_mm` | mm | 光学外缘拟合半径，未补偿倒角与光学散射。 |

| `rms_residual_um` | μm | 各视图有效边缘点径向均方残差的八方向等权平方根。 |

| `max_residual_um` | μm | 全部最终有效圆边点的最大绝对径向残差。 |

| `notch_angle_deg` | ° | notch 相对轴坐标 -Y 的偏角，向 +X 为正；不可用时为 NaN。 |

notch 区域不参与圆拟合。设 N 为缺口中线上的代表点，C 为圆心，角度约定可表示为 `atan2(Nx-Cx, Cy-Ny) * 180/pi`。这不是 die/Mark 排列角度，也不是机械 θ 轴反馈值。

原生函数不包含数组容量参数，不能检测调用方实际分配长度；调用方必须保证八个指针、十六个坐标和每张图像缓冲区均满足合同。

11.3 C# 调用示例

使用 1.3 节中的 `WaferCalibNative.WaferEightViewEdgeCircleResult` 和八图 P/Invoke 声明。以下包装固定八个像素数组，在同步调用结束后释放固定句柄。

```
using System;
using System.Runtime.InteropServices;
using WaferVisionSystem.Interop;

public static class EightViewEdgeCircleDemo {
    public static WaferCalibNative.WaferEightViewEdgeCircleResult Calculate(
        byte[][] images, double[] stageXYmm,
        int width, int height, double pixelSizeUm) {

        if (images == null || images.Length != 8 ||
            stageXYmm == null || stageXYmm.Length != 16)
            throw new ArgumentException("必须输入8张图像及16个XY坐标。");
        if (width < 512 || height < 512 ||
            double.IsNaN(pixelSizeUm) ||
            double.IsInfinity(pixelSizeUm) || pixelSizeUm <= 0)
            throw new ArgumentException("尺寸或像素当量无效。");

        int bytes = checked(width * height);
        for (int i = 0; i < 8; ++i) {
            if (images[i] == null || images[i].Length != bytes)
                throw new ArgumentException("需要紧凑Mono8像素数据。");
        }
        foreach (double value in stageXYmm) {
            if (double.IsNaN(value) || double.IsInfinity(value))
                throw new ArgumentException("轴坐标必须为有限数值。");
        }

        var handles = new GCHandle[8];
        var pointers = new IntPtr[8];
        try {
            for (int i = 0; i < 8; ++i) {
```

```
                handles[i] = GCHandle.Alloc(
                    images[i], GCHandleType.Pinned);
                pointers[i] = handles[i].AddrOfPinnedObject();
            }
            WaferCalibNative.WaferEightViewEdgeCircleResult result;
            int status = WaferCalibNative.Wafer_FindEightViewEdgeCircle(
                pointers, stageXYmm, width, height,
                pixelSizeUm, out result);
            if (status != WaferCalibNative.WAFER_SUCCESS)
                throw new InvalidOperationException(
                    "圆心计算失败，错误码：" + status);
            // 返回0只保证圆心有效，角度使用前还需检查NaN/Infinity。
            return result;
        }
        finally {
            for (int i = 0; i < 8; ++i) {
                if (handles[i].IsAllocated) handles[i].Free();
            }
        }
    }
}
```

业务调用示例（`image0` 至 `image7` 为已解码的同尺寸 Mono8 数组）：

```
var result = EightViewEdgeCircleDemo.Calculate(
    new[] { image0, image1, image2, image3,
            image4, image5, image6, image7 },
    new double[] {
         105.033,  189.6687,
         105.033,  -22.46332,
        -107.099,  189.6687,
        -107.099,  -22.46332,
```

```
         148.967,   83.6027,
        -151.033,   83.6027,
          -1.033,  233.6027,
          -1.033,  -66.3973
    },
    4096, 4096, 1.8); // 2.5X名义换算值，尚未独立标定


bool angleValid = !double.IsNaN(result.NotchAngleDeg)
               && !double.IsInfinity(result.NotchAngleDeg);
Console.WriteLine($"圆心=({result.CenterXmm:F6}, {result.CenterYmm:F6}) mm");
Console.WriteLine($"半径={result.RadiusMm:F6} mm，RMS={result.RmsResidualUm:F3} um");
if (angleValid)
    Console.WriteLine($"notch偏角={result.NotchAngleDeg:F6} deg");
else
    Console.WriteLine("圆心有效，notch角度不可用。");
```

完整可运行的 .NET Framework x64 示例见 `samples/EightViewEdgeCircleSample.cs`。使用含新增导出的 x64 DLL，并保证配套 OpenCV 运行库可被加载；结果结构大小应为 `Marshal.SizeOf(typeof(WaferCalibNative.WaferEightViewEdgeCircleResult)) == 48`。

11.4 notch 定位与错误处理

正常圆边沿用局部归一化阈值、亚像素定位及二次曲线/MAD 异常点剔除。先用 0～6 图拟合参考圆，定位第 7 图向圆内凹陷的 notch 区并加保护带，将其排除后进行八图等权圆拟合。

第 7 图轮廓变换为相对拟合圆的角度与径向深度，在 0.3～1.0 mm 的八个深度截面寻找唯一左右交点，至少三个有效截面时，取截面中线角度的中位数。反光造成的多交点截面及跨越较长缺失区的交点不使用。当前方案适用于已测约 1 mm 深的 notch，更小或严重遮挡的 notch 可能只返回圆心。

| 返回状态 | 结果约定与处理 |

|---|---|

| `0` | 圆心、半径、残差有效；角度须另行检查 NaN/Infinity。 |

| `1` | 参数无效：空指针、尺寸过小、坐标非有限、像素比非法或采集顺序不满足约定。 |

| `4` | 任一方向正常圆边点不足、对比度不足或 notch 排除后正常圆弧不足。 |

| `5` | 圆拟合不收敛、几何无效，或 RMS 超过 `10 * pixel_size_um` μm。 |

| `999` | 内部异常。 |

圆心有效但角度不可辨时返回 0，仅 `notch_angle_deg` 为 NaN。 无 notch、缺口过浅或有效截面不足都可能出现此情况。失败时，非空结果结构的六个字段全部为 NaN；不可只看返回码 0 就直接使用角度。

11.5 DLL 与 C# 实测结果及标定限制

以下为实际调用本版 Release x64 DLL 的结果，非先前离线脚本结果。像素比分别由 5X 的 0.9 μm/px 换算为 1.5X 的 3.0 和 2.5X 的 1.8 μm/px。

| 数据集 | 圆心 X/mm | 圆心 Y/mm | 半径/mm | RMS/μm | notch/° |

|---|---:|---:|---:|---:|---:|

| EdgeImage_1.5X_thick_8 | -0.741851 | 84.207766 | 149.660431 | 18.890 | -0.412041 |

| EdgeImage_2.5X_thick_8 | -1.125728 | 84.325289 | 149.659852 | 11.573 | -0.409433 |

| EdgeImage_2.5X_8 | -1.451286 | 84.192658 | 149.738497 | 12.095 | -0.176680 |

两组 thick 图片同时通过 C# x64 实际调用，六个输出与 ctypes 调用一致。新接口测试覆盖已知圆心、正负已知 notch 角度、无 notch、非法参数、输入只读及第八图参与拟合；旧四图回归通过。证据见 `output/EightViewEdgeCircle/dll_replay.json`、`csharp_replay.csv`、`TEST_REPORT.md`。

本版尚未补偿跨倍率参考点偏移、实测尺度偏差及约 0.28° 的相机轴向偏差，因此两个倍率的圆心仍存在约 0.4 mm 差异。RMS 是边缘到圆的拟合残差，不是圆心绝对误差；角度也未通过独立基准验收。保留小数仅用于复算，不代表达到对应测量精度。

使用本次构建的 `build/Release/WaferCalibSDK.dll`；`docs` 中旧 DLL 不会随说明文件更新而自动替换。

11.6 从四图迁移到八图

| 项目 | 旧四图接口 | 新八图接口 |

|---|---|---|

| 导出名 | `Wafer_FindFourViewEdgeCircle` | `Wafer_FindEightViewEdgeCircle` |

| 图像指针数量 | 4 | 8，固定顺序 0～7 |

| XY 坐标数量 | 8 | 16 |

| 结果结构 | `WaferEdgeCircleResult` | `WaferEightViewEdgeCircleResult` |

| 结构大小 | 40 字节 | 48 字节 |

| 输出 | 圆心、半径、RMS、最大残差 | 原五字段加 notch 角度 |

| 参数个数 | 6 | 6 |

迁移时需同时更新 P/Invoke 导出名、数组数量和结果类型；仅增加图片数量、仍调用旧导出不会启用八图计算。不得给新接口传入 40 字节旧结果结构，否则会发生越界写入。旧导出及旧结果类型保留，已有四图调用可继续使用。

十二、 平场校正与暗场-平场-LUT极速预处理流水线 (FlatField & PreprocessPipeline)

12.1 业务场景与物理链路架构

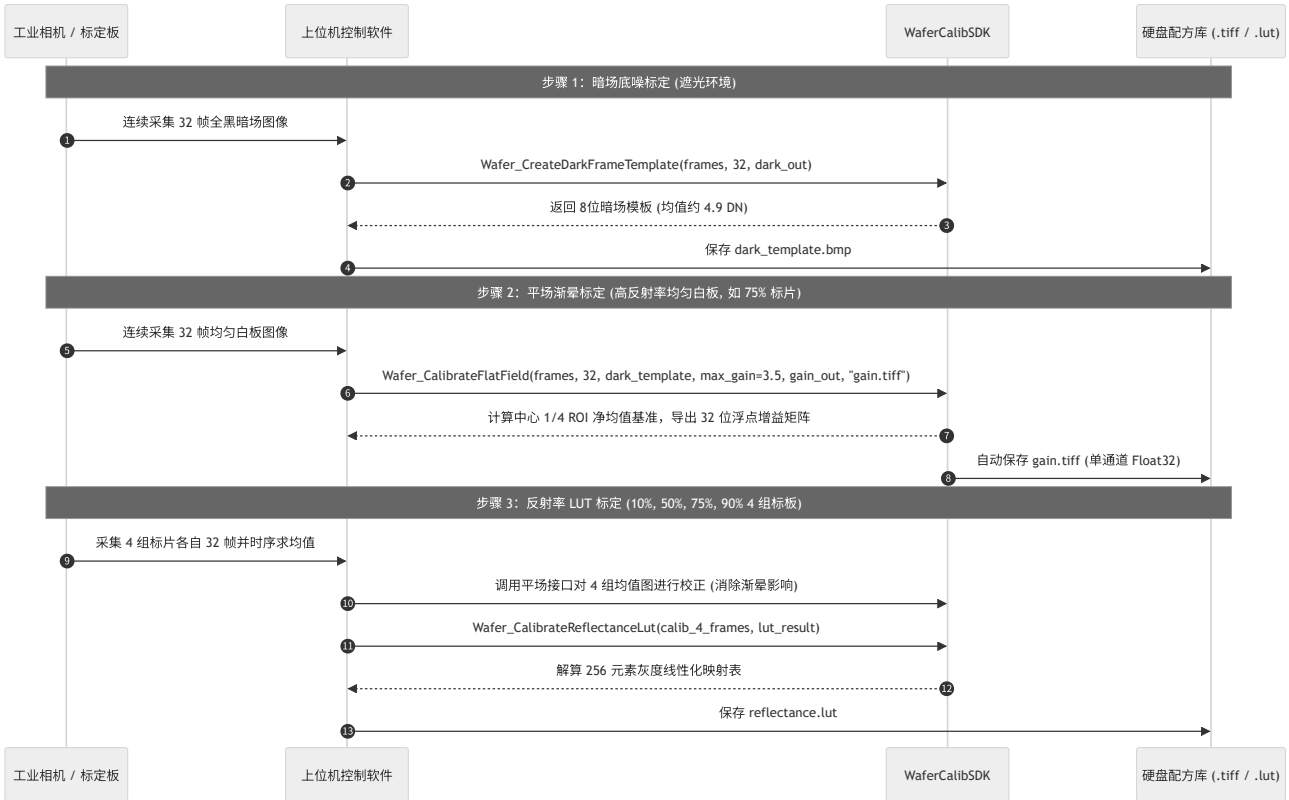
半导体与晶圆检测光学系统中，工业相机采集的原始图像（Raw Image）通常存在以下三类物理畸变与噪声：

1. **暗电流与固定模式噪声 (FPN / Dark Current)**：相机传感器在无光照时由于热激励产生的底噪偏置，导致黑色不纯；
2. **光学渐晕与像元光电响应非均匀性 (Vignetting & PRNU)**：镜头光学通量中心高、边缘低（自然渐晕与余弦四次方定律 $\cos^4\theta$），以及各感光像元量子效率微小差异；
3. **相机传感器光电转换非线性 (Sensor Non-linearity)**：传感器输出灰度值（DN）与晶圆表面物理反射率（Reflectance）之间并非理想线性，难以直接进行薄膜膜厚、缺陷灰度量化。

为了彻底消除上述非理想效应，系统在软件接收到相机图像后，必须进行严格有序的预处理：

12.2 离线标定流程与物理因果逻辑

三个预处理模块的模板标定具有严格的**物理前后因果依赖**。若顺序颠倒或混淆，会导致严重误差：



关键技术要点说明：

1. **为什么平场标定必须拍 32 张图?**

平场针对的是全图 $4096 \times 4096 \approx 1678$ 万个独立像素各自的增益。单个像素受光子随机散粒噪声影响较大。32 帧时序平均将散粒噪声压降 $\sqrt{32} \approx 5.66$ 倍，确保增益矩阵不会将随机噪声固化进配方。

1. **为什么 LUT 标定不需要新增多张接口?**

LUT 是提取全图中心大 ROI（数十万像素）的宏观平均灰度或直方图峰值，空间统计本身具备极强的高斯滤波平滑特性。更重要的是，**LUT 必须映射经过暗场和平场校正后的均匀灰度**；因此在标定时，先对 4 组标片平均图进行平场校正，再输入现有的 `Wafer_CalibrateReflectanceLut` 才是物理严谨的最佳闭环。

12.3 在线流水线处理架构：分步串行 vs 融合单次访存

在产线在线高速运行期间，软件接收到每一帧相机原图，有两种预处理调用模式：

| 模式 | 实现方式 | 4096×4096 单帧耗时 | 吞吐能力 | 特点与适用场景 |

| :--- | :--- | :---: | :---: | :--- |

| **模式 A：两步分步串行** | 1. `Wafer_ApplyFlatFieldCorrection`
2. `Wafer_ApplyLutToImage` | 约 12.9 ms | ~77 fps | 代码完全解耦，支持中间平场调试分析，支持原地就地校正（`out == in`） |

| **模式 B：三合一极速流水线** | 调用 `Wafer_ApplyPreprocessPipeline` | **约** 11.8 ms | **>84 fps** | **单次内存遍历**，彻底消除中间缓冲区与缓存换出，性能极致，100% 位级等价 |

融合流水线核心数学内核：

在单次多核 OpenMP 并行循环中执行：

$$I_{\text{out}}[i] = \text{LUT}\left[\text{clamp}\left((I_{\text{in}}[i] - D[i]) \times \text{Gain}[i] + 0.5\right)\right]$$

若 D（暗场）为 `NULL`，内部自动跳过减法；若 LUT 为 `NULL`，则直接输出平场值。

12.4 核心数据结构与极简接口参数详解

1. 平场标定质量评估结构体 `WaferFlatFieldMetrics`

```
typedef struct WaferFlatFieldMetrics {
    double raw_nu;        // 校正前全图非均匀度 Non-Uniformity (%)
    double corrected_nu;  // 校正后全图非均匀度预测值 (%)
    double vignetting;    // 镜头角隅光学渐晕衰减率 (%)
} WaferFlatFieldMetrics;
```

2. `Wafer_CalibrateFlatField` (平场标定接口)

```
WAFER_API int Wafer_CalibrateFlatField(
```

```
    const unsigned char** flat_buffers,
    int flat_count,
    const unsigned char* dark_template,
    int width,
    int height,
    float max_gain_clamp,
    float* out_gain_map,
    const char* save_template_path,
    WaferFlatFieldMetrics* out_metrics
);
```

- `flat_buffers`：输入平场多帧灰度图像指针数组（推荐 16~32 帧）；
- `flat_count`：平场图像帧数；
- `dark_template`：暗场底噪模板指针（Mono8，可为 `NULL`）；
- `width, height`：图像宽高（如 4096, 4096）；
- `max_gain_clamp`：增益截断上限（默认传入 `3.5f`，防止暗角脏污被极度放大）；
- `out_gain_map`：接收 32 位浮点增益矩阵指针（需由调用方预分配 `width * height * sizeof(float)` 字节）；
- `save_template_path`：配方自动保存路径（支持 `.tiff` 32位图 或 `.bin`，传 `NULL` 或空串表示仅在内存计算）；
- `out_metrics`：输出标定质量指标（可为 `NULL`）。

3. `Wafer_ApplyFlatFieldCorrection` (实时平场校正接口)

```
WAFER_API int Wafer_ApplyFlatFieldCorrection(
    const unsigned char* input_image,
    unsigned char* output_image,
    int width,
    int height,
    const unsigned char* dark_template,
    const float* gain_map
);
```

- `input_image`：待校正原始图像指针；
- `output_image`：校正输出图像指针（**支持原地就地校正：** `output_image == input_image`）；

- `dark_template`：暗场模板指针（传 `NULL` 表示不扣暗场）；
- `gain_map`：32 位浮点平场增益矩阵指针。

4. `Wafer_ApplyPreprocessPipeline` (三合一极速预处理流水线接口)

```
WAFER_API int Wafer_ApplyPreprocessPipeline(
    const unsigned char* input_image,
    unsigned char* output_image,
    int width,
    int height,
    const unsigned char* dark_template,
    const float* gain_map,
    const unsigned char* lut_256
);
```

- `lut_256`：256 元素灰度映射查表指针（若传 `NULL` 则跳过查表直接输出平场灰度）。

12.5 C# P/Invoke 原生互操作与端到端示例

```
using System;
using System.IO;
using System.Runtime.InteropServices;


public static class WaferPreprocessNative
{
    [StructLayout(LayoutKind.Sequential)]
    public struct WaferFlatFieldMetrics
    {
        public double raw_nu;
        public double corrected_nu;
        public double vignetting;
    }
```

```
[DllImport("WaferCalibSDK.dll", CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_CalibrateFlatField(
    IntPtr[] flatBuffers,
    int flatCount,
    IntPtr darkTemplate,
    int width,
    int height,
    float maxGainClamp,
    IntPtr outGainMap,
    [MarshalAs(UnmanagedType.LPUTF8Str)] string saveTemplatePath,
    ref WaferFlatFieldMetrics outMetrics
);

[DllImport("WaferCalibSDK.dll", CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_ApplyFlatFieldCorrection(
    IntPtr inputImage,
    IntPtr outputImage,
    int width,
    int height,
    IntPtr darkTemplate,
    IntPtr gainMap
);

[DllImport("WaferCalibSDK.dll", CallingConvention = CallingConvention.Cdecl)]
public static extern int Wafer_ApplyPreprocessPipeline(
    IntPtr inputImage,
    IntPtr outputImage,
    int width,
    int height,
    IntPtr darkTemplate,
    IntPtr gainMap,
```

```
        IntPtr lut256
    );
}

// 在线相机回调中就地处理示例：
public unsafe void OnCameraFrameArrived(byte* pCamBuffer, int width, int height)
{
    // pCamBuffer: 相机采集直出缓冲区 (Mono8)
    // _pDarkTemplate: 8位暗场模板指针
    // _pGainMap: 32位浮点增益矩阵指针
    // _pLut256: 256字节线性化LUT表指针

    // 原地三合一极速预处理：单次访存直出，耗时仅 ~11.8ms (4096x4096)
    int ret = WaferPreprocessNative.Wafer_ApplyPreprocessPipeline(
        (IntPtr)pCamBuffer,
        (IntPtr)pCamBuffer, // 就地覆盖
        width,
        height,
        (IntPtr)_pDarkTemplate,
        (IntPtr)_pGainMap,
        (IntPtr)_pLut256
    );

    if (ret != 0)
    {
        // 异常处理
    }
}
```

12.6 32位浮点 TIFF 配方存储与指针零损传输规范

- **配方保存**：调用 `Wafer_CalibrateFlatField` 时传入路径如 `D:/recipes/gain_5x.tiff`，SDK 内部直接写入单通道 32 位浮点 TIFF 图像（文件大小约为 `4096 * 4096 * 4 = 67.1 MB`）；
- **配方载入**：在 C# 或 C++ 上位机中，使用标准的 OpenCV 或工业图像库将该 TIFF 读取至单通道 32 位浮点内存中（如 `CV_32FC1` 或 `float[]`）；
- **指针无损传递**：直接获取该内存的首地址指针（`float*`），传入 `Wafer_ApplyPreprocessPipeline`。经 SDK 实测验证，磁盘 TIFF 读取的浮点数组与原内存矩阵的**绝对误差为 0.00000000（位级零损耗）**。

12.7 真实产线实测性能与效果验证

测试数据集：`D:\images\谷神星\标准化\预处理\test\5X（900-5）`（暗场与 1#~4# 标片各 32 帧共 160 张全画幅 4096×4096 图）：

| 标定板反射率 | 标定总耗时 | 原始图像均值 | 原始非均匀度 (NU) | 镜头角隅光学渐晕率 | 增益系数范围 | 配方文件规格 |

| :--- | :---: | :---: | :---: | :---: | :---: | :---: |

| **1# 标片 (90%)** | 1941 ms | 197.64 DN | **3.81 %** | **10.78 %** | [0.941, 1.326] | 64 MB 32位 TIFF |

| **2# 标片 (75%)** | 1441 ms | 166.39 DN | **3.98 %** | **11.13 %** | [0.937, 1.344] | 64 MB 32位 TIFF |

| **3# 标片 (50%)** | 1714 ms | 114.28 DN | **4.24 %** | **11.57 %** | [0.926, 1.344] | 64 MB 32位 TIFF |

| **4# 标片 (10%)** | 1471 ms | 22.84 DN | **5.83 %** | **10.00 %** | [0.553, 3.500] | 64 MB 32位 TIFF |

实测核心指标结论：

1. **光学渐晕物理自洽**：四组不同反射率均匀标定板测得的镜头角隅渐晕衰减率稳定在 $10.0\% \sim 11.6\%$；
2. **单帧图像质量改善**：校正后单帧非均匀度从 3.98% 压降至 1.02%（改善 **3.90 倍**），残余的 1.02% 达到了物理散粒噪声下限；
3. **流水线处理耗时**：4096×4096 全画幅原地处理耗时稳定在 $11.87\ \text{ms}$，极限帧率超 $84\ \text{fps}$，与分步串行处理结果 **100% 位级吻合。**

附录：C++ 原生模块架构参考

针对使用 C++ 直接集成的场景，SDK 内部核心算法均采用面向对象的纯 C++ 设计，头文件位于 `include/wafer_calib/` 目录下：

| 功能大类 | 对应 C++ 头文件 | 核心 C++ 算法类 | 核心方法 |

|---|---|---|---|

| **暗场背景降噪** | `noise_reduction.hpp` | `WaferNoiseReduction` | `CreateDarkFrameTemplate(...)` |

| **四十字 Mark 定位** | `mark_center.hpp` | `WaferMarkCenterDetector` | `FindCenter(...)` |

| **水平标定线角度** | `line_angle.hpp` | `WaferLineAngleDetector` | `FindAngle(...)` |

| **畸变校正与多视野** | `distortion_correction.hpp` | `WaferDistortionCorrection` | `CreateTemplateFromDotGrid(...)`
`CreateTemplateFromMultiViewDotGrid(...)`
`CorrectImage(...)`
`SaveTemplateToFile(...)`
`LoadTemplateFromFile(...)` |

| **圆中心与图像中心差值** | `circle_center_offset.hpp` | `WaferCircleCenterOffsetDetector` | `FindCenterOffset(...)` |

| **平台旋转中心计算** | `rotation_center.hpp` | `WaferPlatformRotationCenterDetector` | `Calculate(...)` |

| **反射率响应与线性化LUT** | `reflectance_lut.hpp` | `WaferReflectanceLutCalibrator` | `Calibrate(...)`
`ApplyLut(...)`
`SaveLut(...)`
`LoadLut(...)` |

| **9点轴-像素标定与坐标转换** | `axis_pixel_calibration.hpp` | `AxisPixelCalibrationModule` | `calibrateGrid(...)`
`calibratePoints(...)`
`transformPixelToAxis(...)`
`transformAxisToPixel(...)`
`saveCalibrationJson(...)`
`loadCalibrationJson(...)` |

| **平场校正与预处理流水线** | `flat_field.hpp` | `FlatFieldModule` | `calibrate(...)`
`apply(...)`
`applyPipeline(...)` |

在 C++ 工程中引入上述头文件并链接 `WaferCalibSDK.lib`，即可直接传入 `cv::Mat` 容器进行高速面向对象调用。